

Hochschule für angewandte Wissenschaften Kempten

Automatisierungstechnik und Robotik

Dynamisches Greifen und automatisierte Sichtprüfung mit einem kollaborativen Roboter

Projektarbeit

Verfasser: Niklas Tollmann, Joshua Junt, Felix Krawczyk, Jeremy Zid

Betreuer: Prof. Dr. Tobias Weiser

Abgabe: 5. Oktober 2026

Kurzfassung

In vielen Automatisierungsanwendungen liegen Bauteile nicht an einer festen Position, sondern werden mit wechselnder Lage bereitgestellt. Gleichzeitig soll ihre Qualität möglichst direkt im Prozess geprüft werden. In dieser Projektarbeit wird dazu ein Demonstrator aufgebaut, der beide Aufgaben verbindet: Ein kollaborativer Roboter vom Typ LARA der Firma Neura Robotics greift ein Bauteil, das an beliebiger Position und in beliebiger Orientierung in einem Arbeitsbereich liegt, führt es einer automatischen Sichtprüfung zu und sortiert es anschließend als Gut- oder Schlechtheil aus.

Das System besteht aus mehreren Modulen, die über fest definierte Schnittstellen miteinander verbunden sind. Die Objekterkennung findet den Arbeitsbereich im Bild einer Kamera über vier ArUco-Marker, erzeugt daraus eine entzerrte Draufsicht und bestimmt Position und Orientierung des Bauteils. Eine abgeschrägte Ecke des Bauteils macht die Orientierung dabei eindeutig. Die Robotersteuerung rechnet diese Lage in eine Greifpose um und führt die Greif- und Ablegebewegungen mit reduzierter Geschwindigkeit innerhalb fester Arbeitsraumgrenzen aus. An der Prüfstation liest die Sichtprüfung mit einer seitlichen Kamera eine fünfstellige Seriennummer mittels Texterkennung, misst den Durchmesser eines Durchgangslochs über eine Ellipsenanpassung und prüft, ob eine Kerbe vorhanden ist. Ein lernbasierter Greifansatz wird als Alternative zur klassischen Objekterkennung entwickelt und kann über dieselbe Schnittstelle eingebunden werden.

Ein zentraler Zustandsautomat steuert den Ablauf aus Erkennen, Greifen, Prüfen und Sortieren, reagiert auf Fehler mit einem Stopp des Roboters und protokolliert jeden Zyklus. Bedient wird das System über eine grafische Oberfläche. Jedes Modul kann mit Platzhaltern, gespeicherten Testbildern, einer Nachbildung der Robotersteuerung oder der realen Hardware betrieben werden, sodass sich die Module unabhängig voneinander entwickeln und schrittweise in Betrieb nehmen lassen. Die eingesetzten Verfahren der Bildverarbeitung sind bewusst einfach und nachvollziehbar gehalten.

Die Erprobung an der realen Anlage umfasst wiederholte Durchläufe mit einem Gutteil in zufälliger Lage sowie gezielt fehlerhafte Bauteile ohne Kerbe, mit abweichendem Lochdurchmesser und ohne Seriennummer.

Ergebnisse der Versuche in ein bis zwei Sätzen ergänzen.

Inhaltsverzeichnis

Kurzfassung	II
Abbildungsverzeichnis	VI
Tabellenverzeichnis	VIII
1 Einleitung	1
1.1 Motivation	1
1.2 Aufgabenstellung	2
1.3 Zielsetzung	2
1.4 Abgrenzung	2
1.5 Aufbau der Arbeit	3
2 Vorprojekt: KI-basierte Roboterbahnen	4
2.1 Aufgabenstellung und Werkzeugumgebung	4
2.2 Kollisionserkennung	5
2.3 Roboter- und Zellsicherheit	6
2.4 Manuelle Bahnplanung	9
2.5 Vorgehen im Projekt	11
2.6 Werkzeuge zur automatischen Bahnplanung	11
2.7 Bedeutung für das vorliegende Projekt	12
3 Grundlagen	14
3.1 Digitale Bildverarbeitung	14
3.1.1 Digitale Bilder	14
3.1.2 Filterung	14
3.1.3 Schwellwertverfahren	14
3.1.4 Morphologische Operationen	15
3.1.5 Konturen und Bildmomente	16
3.2 Farbräume	16
3.3 Kameramodell und Kalibrierung	17
3.4 Fiducial Marker und Homographie	18
3.4.1 ArUco-Marker	18

3.4.2	Homographie	19
3.5	Koordinatentransformationen	19
3.6	Industrierobotik und Greifen	20
3.6.1	Kinematik und Koordinatensysteme	20
3.6.2	Bewegungsarten	20
3.6.3	Greifer und Greifstrategie	21
3.6.4	Kollaborative Roboter	21
3.7	Optische Messtechnik und Prüfmerkmale	22
3.7.1	Beleuchtung	22
3.7.2	Kantendetektion	22
3.7.3	Detektion von Kreisen und Ellipsen	22
3.7.4	Formmerkmale und Template-Matching	23
3.7.5	Umrechnung in metrische Größen	23
3.8	Texterkennung (OCR)	23
3.9	Lernbasiertes Greifen	24
4	Systemkonzept	25
4.1	Anforderungen	25
4.2	Hardwareaufbau	26
4.3	Softwarearchitektur	28
4.3.1	Modulstruktur	28
4.3.2	Ablaufsteuerung	29
4.3.3	Schnittstellen	30
4.4	Koordinatensysteme	30
4.5	Test- und Inbetriebnahmestrategie	31
5	Objekterkennung mit der Top-down-Kamera	33
5.1	Bildaufnahme und Kamerakalibrierung	33
5.2	Detektion des Arbeitsbereichs	34
5.3	Segmentierung des Bauteils	34
5.4	Bestimmung der Pose	35
5.5	Transformation in das Roboterkoordinatensystem	35
5.6	Zwischenfazit	37
6	Robotersteuerung	38
6.1	Roboter und Programmierschnittstelle	38
6.2	Greifpose	39
6.3	Greif- und Ablegesequenz	39
6.4	Feste Posen	40
6.5	Kalibrierung zwischen Kamera und Roboter	40

6.6	Sicherheitskonzept	41
6.7	Inbetriebnahme	41
6.8	Zwischenfazit	41
7	Sichtprüfung	42
7.1	Aufbau der Prüfstation	42
7.2	Lokalisierung des Bauteils und Maßstab	43
7.3	Auslesen der Seriennummer	43
7.4	Vermessung des Lochdurchmessers	44
7.5	Prüfung auf Kerbe	44
7.6	Gut-/Schlecht-Entscheidung	45
7.7	Zwischenfazit	45
8	KI-basiertes Greifen	47
8.1	Konzept	47
8.2	Datensatz und Training	47
8.3	Integration	47
8.4	Zwischenfazit	47
9	Integration und Bedienung	48
9.1	Ablauf eines Zyklus	48
9.2	Konfiguration	48
9.3	Bedienoberfläche	50
9.4	Fehlerbehandlung und Sicherheit im Ablauf	52
9.5	Protokollierung	52
10	Evaluation	54
10.1	Versuchsaufbau und Vorgehen	54
10.2	Ergebnisse Versuchsreihe 1: Wiederholbarkeit	55
10.3	Ergebnisse Versuchsreihe 2: Aussortierung	55
10.4	Vergleich klassischer und KI-basierter Ansatz	55
11	Fazit und Ausblick	56
11.1	Zusammenfassung	56
11.2	Diskussion	56
11.3	Ausblick	56
	Literaturverzeichnis	57

Abbildungsverzeichnis

2.1	Anlegen eines Kollisionssets: Die Objekte links werden gegen die Objekte rechts geprüft	5
2.2	Hervorgehobene Objekte eines Kollisionssets in einer Roboterzelle . . .	6
2.3	Festlegen des Zellbereichs über Eckpunkte der Grundfläche und Höhen- grenzen	7
2.4	Parameter eines Überwachungsraums vom Typ Schutzraum	8
2.5	Darstellung eines Schutzraums (transparenter Quader) um eine Vorrichtung	8
2.6	Festlegen der Werkzeugkugeln über Mittelpunkt und Radius	9
2.7	Greifer mit den umschließenden Werkzeugkugeln	10
2.8	Manuelle Bahnplanung zwischen zwei Vorposen	10
4.1	Aufbau des Demonstrators	26
4.2	Modulstruktur der Software	28
4.3	Zustandsautomat eines Arbeitszyklus. Aus jedem Zustand außer IDLE führt ein Fehler in den Zustand ERROR (nicht eingezeichnet).	29
5.1	Verarbeitungskette der Objekterkennung	33
5.2	Posenbestimmung mit Fasenerkennung. Oben: Kontur, umschließendes Rechteck und Abstand d_k der Rechteckecke an der Fase. Unten: Ablauf; b_R ist die Breite von R , f ein fester Anteil.	36
5.3	Zwischenergebnisse der Objekterkennung	37
6.1	Greif- und Ablegesequenz: Bewegung in der Seitenansicht (links) und Schritte von pick und place (rechts). Die Vorposition V liegt um den Sicherheitsabstand h über der Zielpose Z . Geprüft wird gegen die Ar- beitsraumgrenzen.	40
7.1	Ablauf der Sichtprüfung. Die drei Merkmale werden unabhängig vonein- ander bestimmt und gemeinsam bewertet.	42
7.2	Seitenansicht des Bauteils mit Prüfbereichen	43
7.3	Gut-/Schlecht-Entscheidung. Jedes nicht erfüllte Merkmal wird als Grund in G festgehalten.	46

9.1	Aufrufe des Orchestrators an die Module während eines Arbeitszyklus .	49
9.2	Bedienoberfläche, Reiter „Ablauf“	51

Tabellenverzeichnis

4.1	Funktionale Anforderungen	25
4.2	Nicht-funktionale Anforderungen	26
4.3	Komponenten des Demonstrators	27
4.4	Zustände des Arbeitszyklus und beteiligte Module	29
4.5	Zwischen den Modulen ausgetauschte Datenobjekte	30
4.6	Schnittstellen der Module	30
4.7	Betriebsstufen der Module	31
6.1	Feste Posen des Roboters	40
9.1	Wählbare Varianten der Module	50
9.2	Reiter der Bedienoberfläche	51
10.1	Fehlerhafte Bauteile für Versuchsreihe 2	54

Abkürzungsverzeichnis

CNN Convolutional Neural Network

Cobot kollaborativer Roboter

CSV Comma-Separated Values

GUI Graphical User Interface

HSV Hue, Saturation, Value

KS Koordinatensystem

LAN Local Area Network

LIN Linearbewegung

OCR Optical Character Recognition

PTP Point-to-Point

ROI Region of Interest

TCP Tool Center Point

1 Einleitung

1.1 Motivation

In vielen Automatisierungsanwendungen liegen Bauteile nicht an einer exakt festgelegten Position, sondern werden von Hand, über ein Förderband oder aus einem vorgelagerten Prozess mit wechselnder Lage bereitgestellt. Ein Roboter, der nur fest programmierte Positionen anfährt, kann solche Teile nicht greifen. Mit einer Kamera, die Position und Orientierung des Bauteils bestimmt, lässt sich die Greifbewegung dagegen für jedes Teil neu berechnen. Dieses dynamische Handhaben macht mechanische Vorrichtungen zur exakten Ausrichtung der Teile überflüssig.

Eine zweite Aufgabe, die sich mit Kameras automatisieren lässt, ist die Qualitätsprüfung. Wird jedes Teil direkt im Prozess geprüft, statt nur stichprobenartig am Ende, lassen sich fehlerhafte Teile sofort aussortieren. Werden Handhabung und Prüfung in einer Zelle kombiniert, übernimmt der Roboter neben dem Transport auch das Zuführen zur Prüfung und das Sortieren nach dem Ergebnis.

Das vorliegende Projekt verbindet beide Aufgaben in einem Demonstrator. Die Aufgabe hat sich die Projektgruppe selbst gestellt, nachdem ein vorheriges Industrieprojekt zu KI-basierten Roboterbahnen vorzeitig beendet wurde (Kapitel 2). Sie ist bewusst industrienah gewählt: Eingesetzt werden ein realer kollaborativer Industrieroboter mit seiner Herstellerschnittstelle, Kameras und Verfahren der klassischen Bildverarbeitung, die bewusst einfach und nachvollziehbar gehalten sind. Im Vordergrund steht, ein durchgängig funktionierendes Gesamtsystem aufzubauen, und nicht, einzelne Verfahren zu optimieren.

Optional: Aussagen zur Bedeutung von flexibler Handhabung und Inline-Prüfung in der Automatisierung mit Quellen belegen (Quellen mit DOI).

Offen: Weiterverwendung im Pflegerobotik-Projekt erwähnen? (Einleitung, Fazit oder Ausblick, siehe Blueprint)

1.2 Aufgabenstellung

Ziel des Projekts ist ein Demonstrator, der ein Bauteil mit beliebiger Position und Orientierung in einem durch vier ArUco-Marker begrenzten Arbeitsbereich erkennt, mit einem Roboter greift und an einer Prüfstation ablegt. Dort wird eine Seriennummer ausgelesen, ein Lochdurchmesser vermessen und das Vorhandensein einer Kerbe geprüft. Abhängig vom Prüfergebnis sortiert der Roboter das Bauteil als Gut- oder Schlechtheil aus.

Das System ist in vier Module gegliedert: die Objekterkennung mit einer Kamera über dem Arbeitsbereich, die Robotersteuerung, die Sichtprüfung mit einer seitlichen Kamera und ein lernbasierter Ansatz zum Greifen, der als Alternative zur klassischen Objekterkennung entwickelt wird. Ein zentraler Ablauf verbindet die Module zu einem Gesamtsystem.

1.3 Zielsetzung

Das Projekt gilt als erfolgreich, wenn der Ablauf aus Erkennen, Greifen, Prüfen und Sortieren vollständig automatisch durchläuft, ohne dass ein Bediener eingreifen muss. Die Prüfmerkmale dienen dabei der Demonstration: Toleranzen und Grenzwerte sind so gewählt, dass die Funktion der Prüfung gezeigt werden kann, und entsprechen keinen Fertigungsvorgaben.

1.4 Abgrenzung

Um den Umfang des Projekts zu begrenzen, gelten folgende Annahmen und Einschränkungen:

- Im Arbeitsbereich liegt jeweils nur ein Bauteil. Das Vereinzeln von Teilen aus einer ungeordneten Menge wird nicht betrachtet.
- Das Bauteil liegt flach auf dem Tisch. Seine Lage ist damit durch die Position in der Tischebene und einen Drehwinkel vollständig beschrieben, die Greifhöhe ist bekannt.
- Die Anlage ist nicht sicherheitszertifiziert. Sie wird nur unter Aufsicht und mit reduzierter Geschwindigkeit betrieben.

- Der lernbasierte Greifansatz ist eine Alternative zur klassischen Objekterkennung und keine Voraussetzung für den Demonstrator.

1.5 Aufbau der Arbeit

Kapitel 2 fasst die Erfahrungen aus dem Vorprojekt zur Bahnplanung in der Robotersimulation zusammen und ordnet ein, was davon für dieses Projekt relevant ist. Kapitel 3 stellt die Grundlagen der verwendeten Verfahren aus Bildverarbeitung, Messtechnik und Robotik vor. In Kapitel 4 wird das Gesamtkonzept mit Anforderungen, Hardwareaufbau und Softwarearchitektur beschrieben. Die Kapitel 5 bis 8 behandeln die Umsetzung der einzelnen Module. Kapitel 9 zeigt, wie die Module zu einem Gesamtsystem zusammengesetzt und bedient werden. Kapitel 10 beschreibt die Erprobung des Systems, und Kapitel 11 fasst die Ergebnisse zusammen.

2 Vorprojekt: KI-basierte Roboterbahnen

Vor dem vorliegenden Projekt hat die Projektgruppe an einer Aufgabenstellung in der Industrie gearbeitet. Bei der GROB-WERKE GmbH & Co. KG sollte das Thema „Erstellung von KI-basierten Roboterpfaden für die Robotersimulation und Inbetriebnahme“ bearbeitet werden. Nach einer Einarbeitung von acht Tagen vor Ort wurde das Projekt bearbeitet. Während der Bearbeitung wurde die Zusammenarbeit beendet, und die Gruppe hat sich die Aufgabe dieser Arbeit selbst gestellt (Kapitel 1). Die Ergebnisse des Vorprojekts sind deshalb begrenzt.

Dieses Kapitel fasst zusammen, was bis dahin erarbeitet wurde. Es beschreibt die Aufgabe, die Werkzeuge und das Vorgehen bei der Bahnplanung in der Robotersimulation und die ersten Erfahrungen mit Werkzeugen zur automatischen Bahnplanung. Am Ende wird eingeordnet, welche Erkenntnisse für den Demonstrator relevant sind oder eine mögliche Erweiterung wären.

2.1 Aufgabenstellung und Werkzeugumgebung

Bevor eine Roboterzelle aufgebaut wird, werden ihre Bewegungsabläufe in einer Robotersimulation geplant und geprüft. Dafür wird ein digitales Modell der Zelle aus den Konstruktionsdaten verwendet, mit Robotern, Greifern, Vorrichtungen und Schutzeinrichtungen. Die dort erstellten Bahnen werden später auf die realen Roboter übertragen. Je genauer die Bahnen in der Simulation stimmen, desto weniger muss bei der Inbetriebnahme an der Anlage nachgebessert werden.

Bei GROB wird dafür Siemens Process Simulate eingesetzt. Die Bahnen werden dort bisher von Hand erstellt. Das kostet bei großen Zellen mit mehreren Robotern viel Zeit. Ziel des Vorprojekts war es zu untersuchen, ob sich Bahnen mit KI-basierten Werkzeugen automatisch erzeugen lassen und ob diese Bahnen den Anforderungen von GROB genügen. Ergebnis sollte in jedem Fall eine kollisionsfreie und auf die Taktzeit optimierte Roboterbahn sein.

Die Aufgabe wurde auf einen Teil einer bestehenden Zelle eingegrenzt. Betrachtet wurde ein Handhabungsprozess, bei dem ein Roboter Teile zwischen festen Positionen umsetzt. Prozesse wie das Kleben wurden nicht betrachtet. Sie sind deutlich komplexer und für die Fragestellung nicht notwendig. Verwendet wurde ein KUKA-Roboter, weil sich dieser gut in Process Simulate abbilden lässt und einfacher zu handhaben ist.

In der Einarbeitung wurden vier Themen behandelt: Kollisionserkennung, Roboter- und Zellsicherheit, das manuelle Erstellen von Bahnen und erste Werkzeuge zur automatischen Bahnplanung. Sie werden in den folgenden Abschnitten beschrieben, bevor Abschnitt 2.5 das Vorgehen im Projekt zeigt.

Quelle für Siemens Process Simulate (Herstellerdokumentation) ergänzen, falls gewünscht.

2.2 Kollisionserkennung

Die Simulation prüft nicht automatisch jedes Bauteil der Zelle gegen jedes andere. Stattdessen legt der Anwender *Kollisionssets* an. Ein Kollisionsset besteht aus zwei Listen von Objekten: Die Objekte der ersten Liste werden gegen die Objekte der zweiten Liste geprüft (Abb. 2.1). Typischerweise stehen in der ersten Liste der Roboter und sein Greifer. In der zweiten Liste stehen die Vorrichtungen, Werkstücke und Anlagenteile, mit denen er zusammenstoßen könnte. Paare, die sich ohnehin berühren, etwa der Greifer und der Roboterflansch, werden so von der Prüfung ausgenommen. Das spart Rechenzeit und vermeidet Fehlmeldungen.

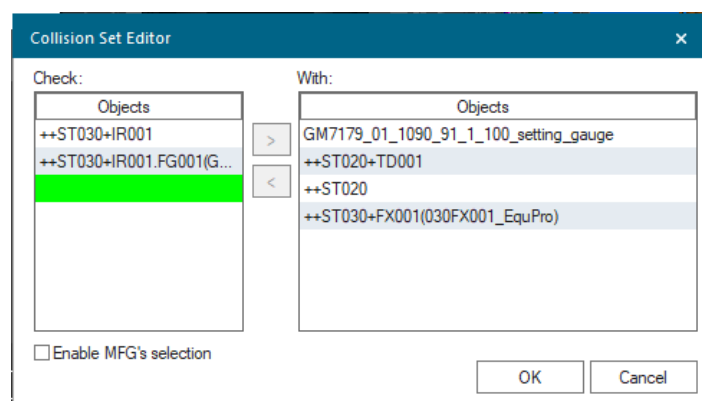


Abbildung 2.1: Anlegen eines Kollisionssets: Die Objekte links werden gegen die Objekte rechts geprüft

Für jedes Kollisionsset lassen sich zwei Grenzwerte einstellen. Der *Beinahe-Abstand* („Near Miss“) meldet, wenn sich zwei Objekte näher als ein festgelegter Abstand kommen, ohne sich zu berühren. Die *zulässige Durchdringung* erlaubt geringe Überschneidungen, zum Beispiel wenn ein Greifer ein Werkstück greift. Während der Simulation hebt die

Software die Objekte eines Sets farbig hervor und listet die kollidierenden Paare auf (Abb. 2.2).

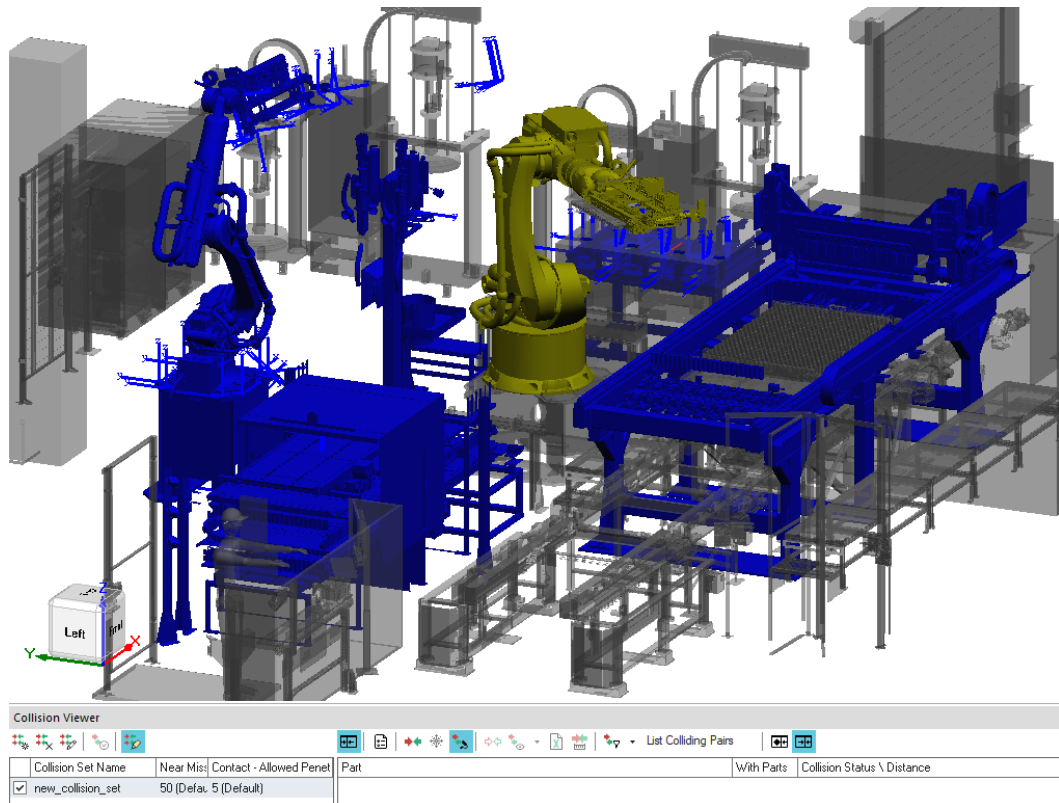


Abbildung 2.2: Hervorgehobene Objekte eines Kollisionssets in einer Roboterzelle

Für die automatische Bahnplanung sind die Kollisionssets die Grundlage. Ein Planungs-
werkzeug kann nur solche Kollisionen vermeiden, die es auch prüft. Fehlt ein Objekt
im Set, kann eine Bahn in der Simulation kollisionsfrei erscheinen und an der realen
Anlage trotzdem zu einem Zusammenstoß führen. Jedes Werkzeug zur automatischen
Bahnplanung muss die Kollisionssets deshalb berücksichtigen.

2.3 Roboter- und Zellsicherheit

Neben der Kollisionsprüfung in der Simulation überwacht die Sicherheitssteuerung des
realen Roboters, dass er bestimmte Bereiche nicht verlässt oder nicht betritt. Diese
Bereiche werden ebenfalls in der Simulation festgelegt und dort sichtbar gemacht. In
der Einarbeitung wurde dafür eine Erweiterung von Process Simulate für die Sicher-
heitskonfiguration der eingesetzten KUKA-Roboter verwendet. Drei Elemente wurden
behandelt.

Zellbereich. Der Zellbereich beschreibt den Raum, den der Roboter insgesamt nicht verlassen darf. Er wird als Prisma angegeben: eine Grundfläche aus bis zu zehn Eckpunkten in der Ebene und eine untere und obere Höhengrenze (Abb. 2.3).

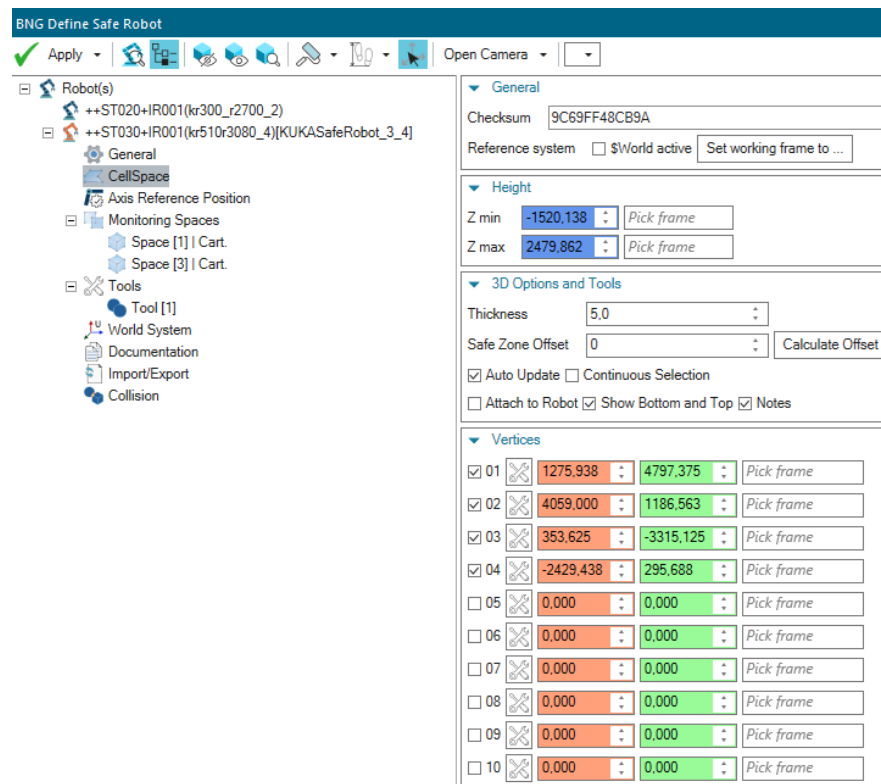


Abbildung 2.3: Festlegen des Zellbereichs über Eckpunkte der Grundfläche und Höhengrenzen

Überwachungsräume. Innerhalb des Zellbereichs lassen sich weitere Räume als Quader festlegen (Abb. 2.4). Ein Quader wird durch ein Bezugskoordinatensystem und die minimalen und maximalen Abstände in jeder Achse beschrieben. Für jeden Raum wird eingestellt, ob der Roboter ihn nicht verlassen darf (Arbeitsraum) oder nicht betreten darf (Schutzraum), ob er immer oder nur unter Bedingungen aktiv ist und ob der Roboter an der Grenze anhält. Ein Schutzraum wird zum Beispiel um eine Vorrichtung gelegt, in die der Roboter nicht eingreifen soll (Abb. 2.5).

Werkzeugkugeln. Damit die Sicherheitssteuerung nicht nur den Tool Center Point (TCP), sondern das ganze Werkzeug überwachen kann, wird der Greifer durch Kugeln angenähert (Abb. 2.6). Jede Kugel ist durch ihren Mittelpunkt relativ zum Roboterflansch und ihren Radius festgelegt. Die Kugeln müssen den Greifer vollständig umschließen. Zu große Kugeln schränken den nutzbaren Arbeitsraum dagegen unnötig ein. Die Steuerung prüft dann für jede Kugel, ob sie einen Überwachungsraum verletzt.

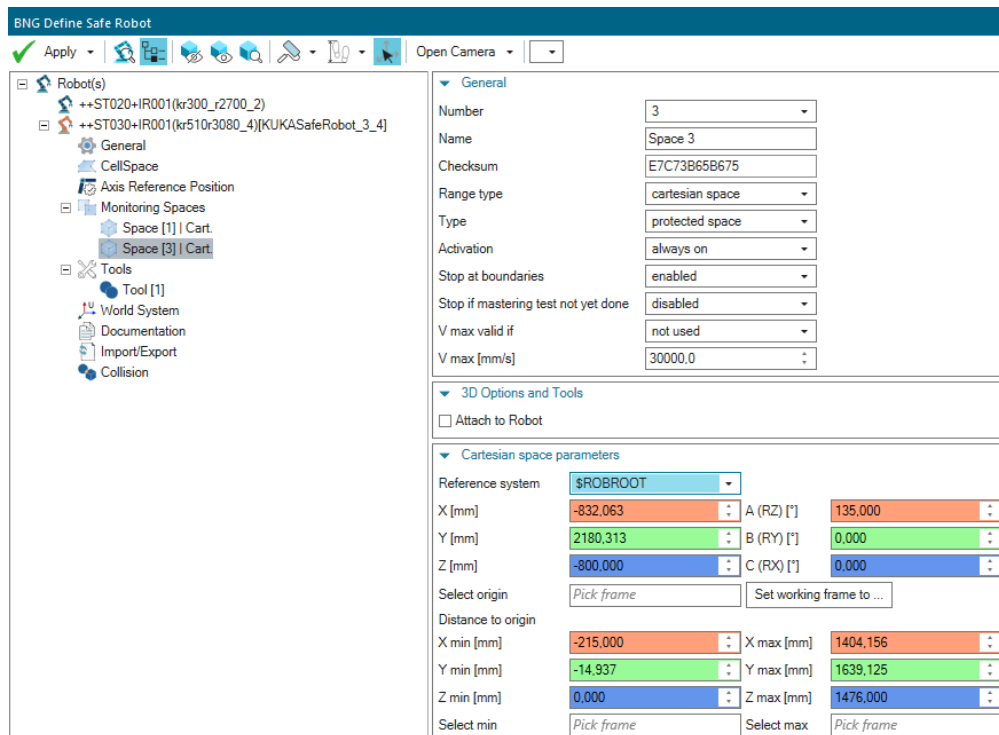


Abbildung 2.4: Parameter eines Überwachungsraums vom Typ Schutzraum

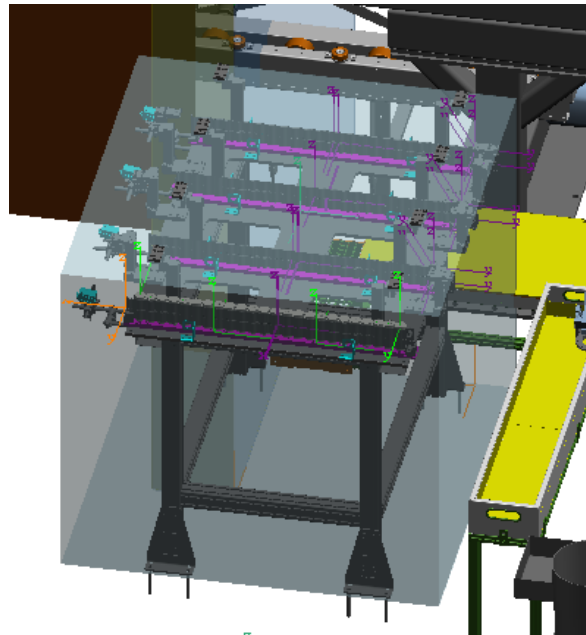


Abbildung 2.5: Darstellung eines Schutzraums (transparenter Quader) um eine Vorrichtung

Kugeln sind dafür gut geeignet, weil sich der Abstand einer Kugel zu einer Fläche mit wenig Rechenaufwand bestimmen lässt.

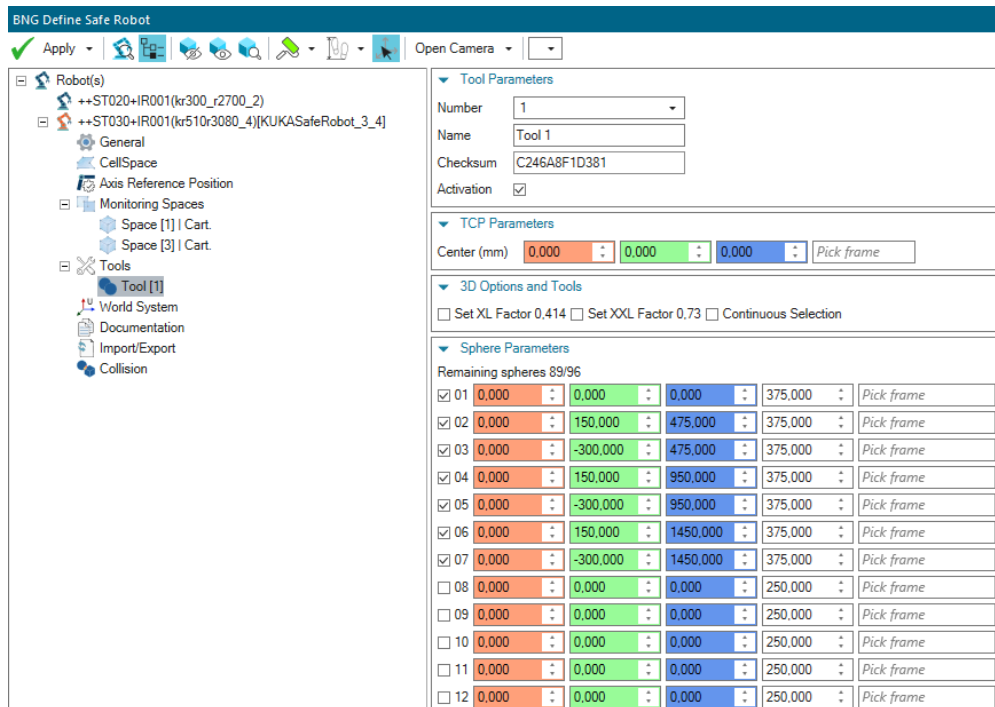


Abbildung 2.6: Festlegen der Werkzeugkugeln über Mittelpunkt und Radius

2.4 Manuelle Bahnplanung

Bei GROB werden die Bahnen nach einem festen Vorgehen erstellt. Gegeben sind die Startpose und die Zielpose, zum Beispiel eine Entnahme- und eine Ablageposition. Zu beiden Posen wird zuerst eine Vorpose erzeugt. Von der Vorpose aus fährt der Roboter mit einer Linearbewegung (LIN) in die Start- bzw. Zielpose, wie es auch in Abschnitt 3.6 beschrieben ist.

Die Verbindung zwischen den beiden Vorposen wird schrittweise aufgebaut (Abb. 2.8). Der Roboter wird in der Simulation in Gelenkkkoordinaten von der einen Vorpose zur anderen verfahren, bis eine Kollision auftritt. An dieser Stelle wird er aus der Kollision herausbewegt, möglichst durch das Verfahren nur einer Achse. Die so gefundene Stellung wird als Zwischenpunkt gespeichert, und die Bahn wird von dort aus erneut betrachtet. Das wird wiederholt, bis die gesamte Bahn kollisionsfrei ist.

Ziel dieses Vorgehens ist, die Bewegungen der einzelnen Achsen klein zu halten. Oft wurde außerdem gewünscht, dass die Achsen 4 und 6 bei 0° bleiben. Das ist nicht zwingend, vermeidet aber unnötige Drehungen der Handachsen und damit eine starke

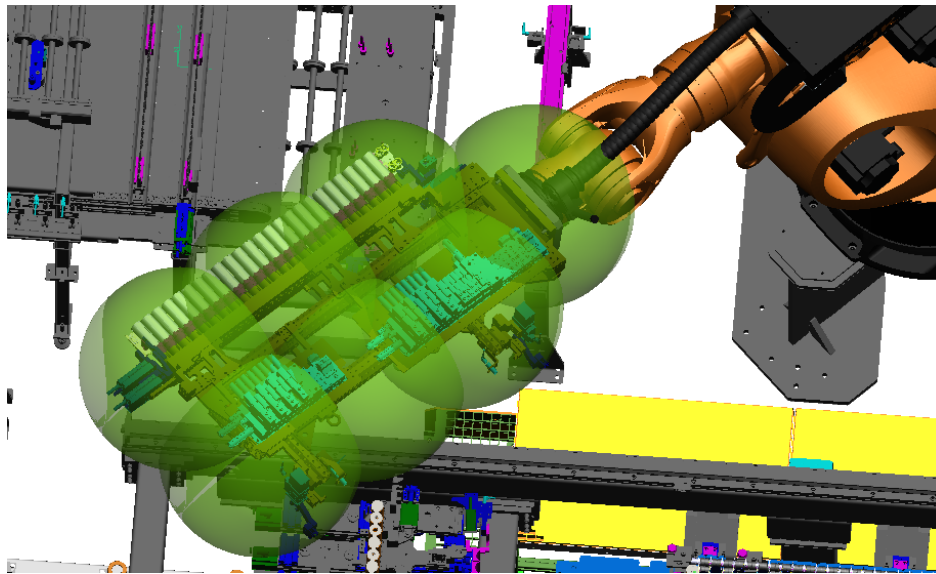


Abbildung 2.7: Greifer mit den umschließenden Werkzeugkugeln

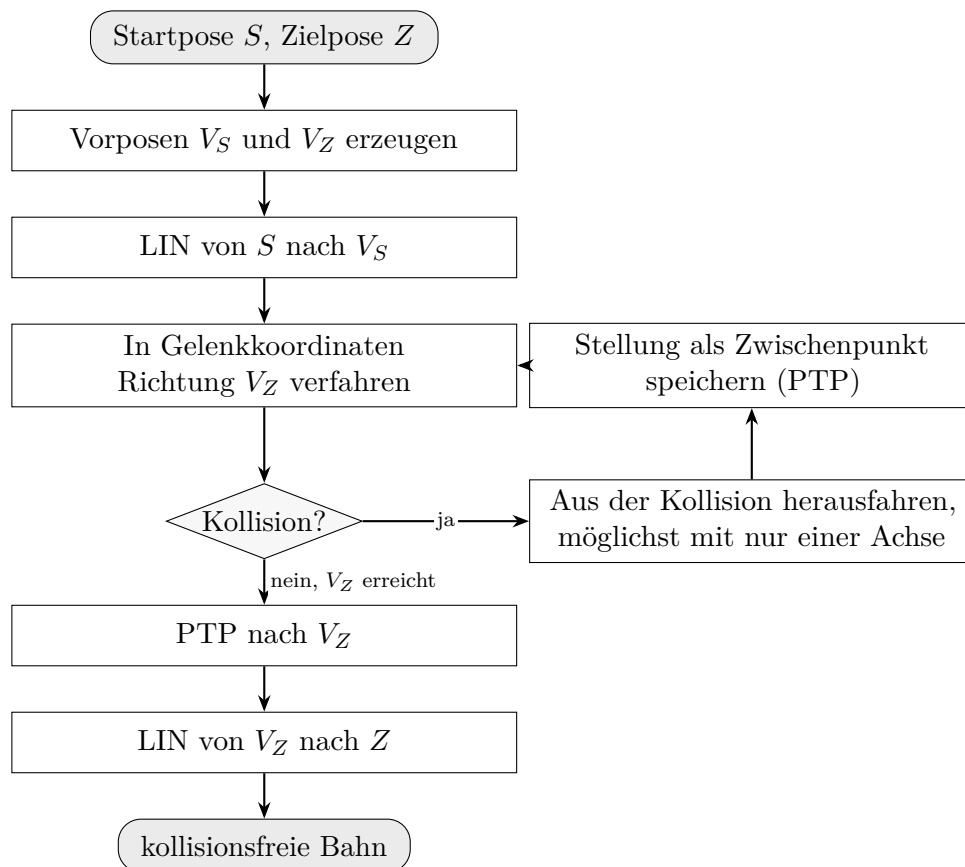


Abbildung 2.8: Manuelle Bahnplanung zwischen zwei Vorposen

Belastung der Energiezuführung zum Greifer. Außerdem bleiben die Bahnen für den Inbetriebnehmer nachvollziehbar.

2.5 Vorgehen im Projekt

Die betrachtete Zelle ist bereits seit einigen Jahren umgesetzt. Deshalb wurde zuerst das bestehende Projekt analysiert. Die Taktzeiten der dort umgesetzten Bahnen dienen als Vergleichsbasis für alle neu erzeugten Bahnen.

Anschließend wurde ein leeres Bahnmodul angelegt. Es enthält nur die Start- und Zielposen und die zugehörigen Vorposen, aber keine Zwischenpunkte. Alle Werkzeuge zur automatischen Bahnplanung erhalten dasselbe Bahnmodul als Ausgangspunkt. Dadurch sind ihre Ergebnisse untereinander und mit der bestehenden Bahn vergleichbar.

Zusätzlich hat die Projektgruppe selbst Bahnen nach dem manuellen Vorgehen aus Abschnitt 2.4 erstellt. Diese Bahnen zeigen, welches Ergebnis mit wenig Erfahrung in der Bahnplanung erreicht wird, und dienen als weiterer Vergleich. Gleichzeitig hat die Gruppe dabei das Programm und die Bahnplanung selbst kennengelernt.

Damit ergeben sich drei Vergleichswerte für eine automatisch erzeugte Bahn: die bestehende, von erfahrenen Planern erstellte Bahn, die eigene manuelle Bahn und die Bahnen der anderen Werkzeuge.

Falls vorhanden: Taktzeiten der bestehenden Bahn, der eigenen manuellen Bahn und der Bahn aus dem Robotic Automatic Path Planner als Tabelle ergänzen.

2.6 Werkzeuge zur automatischen Bahnplanung

Für die automatische Erzeugung der Bahnen waren zwei Werkzeuge vorgesehen. Als dritte Möglichkeit sollten Verfahren betrachtet werden, die nicht in Process Simulate enthalten sind.

Robotic Automatic Path Planner. Der Robotic Automatic Path Planner ist in Process Simulate integriert. Er erzeugt aus einer vorhandenen Bahn mit Start- und Zielpose eine kollisionsfreie Bahn und berücksichtigt dabei die Kollisionssets. Der Anwender kann zwischen mehreren Optimierungszielen wählen, zum Beispiel einer möglichst kurzen Taktzeit.

Das Werkzeug wurde auf das leere Bahnmodul angewendet, und Bahnen konnten erfolgreich automatisch erstellt werden. Bei der Optimierung auf die Taktzeit fiel jedoch auf, dass das Werkzeug auch die programmierten Geschwindigkeiten ändert. Betroffen war zum Beispiel die geradlinige Bewegung von der Vorpose in die Zielpose. Diese Bewegung wird bewusst langsam programmiert, damit das Werkstück sicher abgelegt oder gegriffen wird. Wird sie beschleunigt, kann das Werkstück verrutschen, oder Vorrichtung und Greifer werden stärker belastet. Eine automatisch erzeugte Bahn muss deshalb darauf geprüft werden, ob solche festen Vorgaben erhalten bleiben.

Weitere Optimierungsziele des Robotic Automatic Path Planner nennen (z. B. Abstand zu Hindernissen, Achsbewegung), falls bekannt.

Realtime Robotics. Als zweites Werkzeug war eine Lösung der Firma Realtime Robotics vorgesehen. Die Lizenz stand bis zum Ende der Zusammenarbeit nicht zur Verfügung, deshalb konnte das Werkzeug nicht getestet werden.

Alternative Verfahren.

Kommilitone: Alternative Verfahren zur Bahnplanung außerhalb von Process Simulate beschreiben (Ansätze, betrachtete Werkzeuge, Einschätzung). Neue Quellen nur mit geprüfter DOI.

2.7 Bedeutung für das vorliegende Projekt

Die Aufgabe des vorliegenden Projekts unterscheidet sich vom Vorprojekt. Im Mittelpunkt steht nicht die Planung von Bahnen in der Simulation, sondern das Erkennen, Greifen und Prüfen eines Bauteils am realen Roboter. Mehrere Erkenntnisse aus dem Vorprojekt sind trotzdem übertragbar:

Vorpose und geradlinige Anfahrt Die Greif- und Ablegesequenz des Demonstrators folgt demselben Muster wie bei GROB: Eine Vorpose wird mit einer Gelenkbewegung angefahren, das Ziel dann langsam und geradlinig (Abschnitt 6.3). Die Erfahrung mit dem Robotic Automatic Path Planner zeigt, dass diese langsame Anfahrt als feste Vorgabe gelten muss, auch wenn Bahnen später automatisch erzeugt oder optimiert werden.

Sichere Bereiche Die Robotersteuerung des Demonstrators prüft vor jeder Bewegung, ob Zielpose und Vorposition in einem festgelegten Quader liegen (Abschnitt 6.6). Das entspricht in vereinfachter Form einem Arbeitsraum. Geprüft wird dabei nur der TCP. Eine mögliche Erweiterung wäre, Greifer und Bauteil wie bei

den Werkzeugkugeln durch Kugeln anzunähern und zusätzlich Schutzräume um Kameras und Prüfstation festzulegen.

Kollisionsprüfung in der Simulation Im Demonstrator werden die Posen geteacht, und die Wege dazwischen führen durch freien Raum über dem Tisch. Eine Kollisionsprüfung ist deshalb nicht nötig. Würde die Zelle erweitert, könnten die Bahnen vorab in einer Simulation mit einem Zellmodell und Kollisionssets geprüft werden. Für den eingesetzten Roboter bietet der Hersteller dafür eine Offline-Simulation an [1].

Randbedingungen für KI-Verfahren Im Vorprojekt sollte eine KI die Bahnen erzeugen, im Demonstrator bestimmt ein lernbasiertes Verfahren optional die Greifpose (Kapitel 8). In beiden Fällen gilt: Das Ergebnis der KI muss die Randbedingungen der Anlage einhalten, also kollisionsfrei sein, in den erlaubten Bereichen liegen und feste Vorgaben wie Geschwindigkeiten beibehalten. Im Demonstrator prüft deshalb die Robotersteuerung jede Zielpose gegen die Arbeitsraumgrenzen, unabhängig davon, welches Modul sie liefert.

3 Grundlagen

Dieses Kapitel stellt die Verfahren vor, auf denen die Umsetzung in den Kapitel 5 bis 7 aufbaut. Es beschreibt dabei auch Alternativen, gegen die sich die Umsetzung entschieden hat; die jeweilige Begründung folgt in den Modulkapiteln.

3.1 Digitale Bildverarbeitung

3.1.1 Digitale Bilder

Ein digitales Grauwertbild ist eine Matrix $I(u, v)$, deren Elemente (Pixel) die Helligkeit an der Stelle mit Spalte u und Zeile v angeben, üblicherweise als ganze Zahl von 0 (schwarz) bis 255 (weiß). Ein Farbbild besteht aus drei solchen Matrizen, den Farbkanälen. Der Ursprung liegt in der linken oberen Ecke, die v -Achse zeigt nach unten. Für die Verarbeitung wird im Projekt die Bibliothek OpenCV [2] eingesetzt, die die im Folgenden beschriebenen Verfahren bereitstellt.

3.1.2 Filterung

Bildrauschen erschwert viele nachfolgende Schritte. Es wird meist durch eine Glättung verringert, bei der jeder Pixel durch einen gewichteten Mittelwert seiner Nachbarschaft ersetzt wird. Mathematisch ist das eine Faltung des Bildes mit einem Filterkern. Beim Gaußfilter folgen die Gewichte einer zweidimensionalen Normalverteilung, sodass nahe Pixel stärker eingehen als entfernte. Die Glättung verringert das Rauschen, macht aber auch Kanten unschärfer; die Größe des Filterkerns ist deshalb ein Kompromiss.

3.1.3 Schwellwertverfahren

Eine Segmentierung trennt Bildbereiche, die zu einem Objekt gehören, vom Hintergrund. Das einfachste Verfahren ist ein Schwellwert T : Pixel mit $I(u, v) > T$ werden dem

Objekt zugeordnet, alle anderen dem Hintergrund. Das Ergebnis ist ein Binärbild (Maske).

Ein fester Schwellwert ist empfindlich gegenüber Änderungen der Beleuchtung. Das Verfahren von Otsu [3] bestimmt den Schwellwert deshalb automatisch aus dem Histogramm des Bildes. Es wählt denjenigen Wert T , der die Varianz zwischen den beiden entstehenden Klassen maximiert:

$$\sigma_B^2(T) = \omega_0(T) \omega_1(T) \left(\mu_0(T) - \mu_1(T) \right)^2. \quad (3.1)$$

Dabei sind ω_0 und ω_1 die Anteile der Pixel unterhalb und oberhalb von T und μ_0, μ_1 deren mittlere Grauwerte. Das Verfahren funktioniert gut, wenn das Histogramm zwei deutlich getrennte Häufungen aufweist, also Objekt und Hintergrund ähnlich große Flächen einnehmen und sich klar unterscheiden. Nimmt das Objekt nur einen kleinen Teil des Bildes ein, ist seine Häufung im Histogramm schwach ausgeprägt, und der Schwellwert orientiert sich überwiegend an den Schwankungen des Hintergrunds.

Neben globalen Verfahren gibt es adaptive Schwellwerte, die für jeden Pixel aus seiner lokalen Umgebung berechnet werden, sowie Schwellwerte, die relativ zu einer bekannten Referenz wie der Helligkeit des Hintergrunds festgelegt werden.

3.1.4 Morphologische Operationen

Binärmasken nach einer Schwellwertbildung enthalten oft einzelne Störpixel oder kleine Lücken. Morphologische Operationen bearbeiten die Form der Bereiche mithilfe eines Strukturelements, etwa eines kleinen Quadrats:

- Die **Erosion** setzt einen Pixel nur dann auf Objekt, wenn das gesamte Strukturelement an dieser Stelle im Objekt liegt; Bereiche schrumpfen.
- Die **Dilatation** setzt einen Pixel auf Objekt, sobald das Strukturelement irgendwo das Objekt berührt; Bereiche wachsen.
- Das **Öffnen** (Erosion, dann Dilatation) entfernt Strukturen, die kleiner als das Strukturelement sind, und erhält die übrigen weitgehend in ihrer Form.
- Das **Schließen** (Dilatation, dann Erosion) füllt Lücken und Einbuchtungen, die kleiner als das Strukturelement sind.

3.1.5 Konturen und Bildmomente

Die zusammenhängenden Bereiche einer Binärmaske lassen sich durch ihre Randlinien, die Konturen, beschreiben. OpenCV bestimmt sie mit dem Randverfolgungsverfahren nach Suzuki und Abe [4], das auch die Verschachtelung von äußeren Rändern und Löchern liefert. Aus einer Kontur lassen sich Fläche, Umfang und umschließende Formen wie das kleinste umschließende Rechteck oder der kleinste umschließende Kreis berechnen.

Lage und Ausrichtung eines Bereichs werden über Bildmomente beschrieben [5]. Für einen Bereich mit den Pixeln (u, v) sind die Momente

$$m_{pq} = \sum_u \sum_v u^p v^q I(u, v), \quad (3.2)$$

wobei $I(u, v)$ in einer Binärmaske 1 für Objektpixel und 0 sonst ist. Das Moment m_{00} ist die Fläche, und der Flächenschwerpunkt ergibt sich zu

$$\bar{u} = \frac{m_{10}}{m_{00}}, \quad \bar{v} = \frac{m_{01}}{m_{00}}. \quad (3.3)$$

Die zentralen Momente zweiter Ordnung μ_{20} , μ_{02} und μ_{11} werden auf den Schwerpunkt bezogen berechnet und beschreiben die Streuung des Bereichs. Aus ihnen folgt der Winkel der Hauptachse

$$\theta = \frac{1}{2} \operatorname{atan2}(2\mu_{11}, \mu_{20} - \mu_{02}). \quad (3.4)$$

Die Hauptachse und die lange Seite des kleinsten umschließenden Rechtecks geben bei länglichen Objekten die Orientierung an, allerdings nur bis auf 180° : Eine Achse hat keine Richtung. Für eine eindeutige Orientierung braucht es ein zusätzliches, asymmetrisches Merkmal des Objekts.

3.2 Farbräume

Kameras liefern Farbbilder meist im RGB-Farbraum mit je einem Kanal für Rot, Grün und Blau. In diesem Raum sind Farbe und Helligkeit miteinander verknüpft: Wird ein Objekt heller beleuchtet, steigen alle drei Kanäle, obwohl sich seine Farbe nicht ändert. Für eine Segmentierung nach Farbe ist das ungünstig.

Der Hue, Saturation, Value (HSV)-Farbraum [6] trennt diese Eigenschaften. Er beschreibt eine Farbe durch drei Größen:

Farbton (H) die Farbart als Winkel auf einem Farbkreis, z. B. Rot, Gelb, Grün oder Violett;

Sättigung (S) wie rein die Farbe ist, von grau ($S = 0$) bis zur reinen Farbe;

Hellwert (V) die Helligkeit.

Mit $M = \max(R, G, B)$ und $m = \min(R, G, B)$ gilt

$$V = M, \quad S = \begin{cases} (M - m)/M, & M > 0 \\ 0, & M = 0 \end{cases} \quad (3.5)$$

und der Farbton ergibt sich daraus, welcher Kanal am größten ist und wie sich die beiden anderen zueinander verhalten. In OpenCV wird der Farbton bei 8-Bit-Bildern auf den Bereich 0 bis 180 abgebildet, Sättigung und Hellwert auf 0 bis 255.

Für die Bildverarbeitung ergeben sich daraus zwei Vorteile. Ein farbiges Objekt lässt sich über ein Intervall des Farbtons finden, das weitgehend unabhängig von der Beleuchtung ist. Und unbunte Flächen wie eine graue Platte oder ein weißes Etikett unterscheiden sich von farbigen Objekten durch ihre geringe Sättigung, auch wenn sie ähnlich hell sind.

3.3 Kameramodell und Kalibrierung

Die Abbildung eines Punkts im Raum auf das Kamerabild wird meist mit dem Lochkameramodell beschrieben [7]. Ein Punkt $\mathbf{X} = (X, Y, Z)^\top$ im Welt-Koordinatensystem (KS) wird auf den Bildpunkt (u, v) abgebildet gemäß

$$\lambda \begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = \mathbf{K} [\mathbf{R} \mid \mathbf{t}] \begin{pmatrix} X \\ Y \\ Z \\ 1 \end{pmatrix}, \quad \mathbf{K} = \begin{pmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix}. \quad (3.6)$$

Die *extrinsischen* Parameter $\mathbf{R}$ und $\mathbf{t}$ beschreiben Lage und Ausrichtung der Kamera im Raum. Die *intrinsischen* Parameter in der Kameramatrix $\mathbf{K}$ sind die Brennweiten f_x, f_y in Pixeln und der Hauptpunkt (c_x, c_y) , an dem die optische Achse die Bildebene trifft. Der Faktor λ berücksichtigt, dass alle Punkte auf einem Sehstrahl auf denselben Bildpunkt fallen.

Reale Objektive weichen von diesem Modell ab. Am stärksten wirkt die radiale Verzerrung, bei der gerade Linien zum Bildrand hin gekrümmt erscheinen. Sie wird durch

ein Polynom im Abstand r von der Bildmitte modelliert,

$$x_{\text{verz}} = x(1 + k_1 r^2 + k_2 r^4 + k_3 r^6), \quad (3.7)$$

und analog für y ; hinzu kommen kleinere tangentielle Anteile.

Bei der Kalibrierung werden die intrinsischen Parameter und die Verzeichnungskoeffizienten bestimmt. Verbreitet ist das Verfahren nach Zhang [8]: Ein ebenes Muster mit bekannter Geometrie, etwa ein Schachbrett, wird aus mehreren Blickwinkeln aufgenommen. Aus den Punktkorrespondenzen jeder Aufnahme wird zunächst eine geschlossene Lösung berechnet und diese anschließend durch eine nichtlineare Optimierung des Rückprojektionsfehlers verfeinert. Mit den ermittelten Parametern lässt sich jedes Bild der Kamera entzerren.

3.4 Fiducial Marker und Homographie

3.4.1 ArUco-Marker

Fiducial Marker sind künstliche Muster, die in einem Bild zuverlässig gefunden und eindeutig identifiziert werden können. ArUco-Marker [9] sind quadratisch und bestehen aus einem schwarzen Rand und einem inneren Gitter aus schwarzen und weißen Feldern, das eine Kennung codiert. Die zulässigen Codes sind in Wörterbüchern zusammengefasst; ein Wörterbuch wie `DICT_4X4_50` enthält 50 Marker mit 4×4 Datenbits, die so gewählt sind, dass sie sich möglichst stark voneinander unterscheiden. Dadurch lassen sich einzelne falsch gelesene Bits erkennen oder korrigieren.

Die Detektion läuft in mehreren Schritten ab [9]: Das Graustufenbild wird mit einem adaptiven Schwellwert binarisiert, Konturen werden gesucht und durch Polygone angenähert, und nur konvexe Vierecke bleiben als Kandidaten übrig. Jeder Kandidat wird perspektivisch auf ein Quadrat entzerzt, in das Gitter der Marker unterteilt und bitweise ausgelesen. Stimmt das Bitmuster, gegebenenfalls in einer der vier Drehlagen, mit einem Marker des Wörterbuchs überein, gilt er als erkannt. Die Ecken des Markers sind dann im Bild subpixelgenau bekannt, und aus der Drehlage ergibt sich, welche Ecke welche ist.

Eine Voraussetzung ist, dass der schwarze Rand des Markers sich klar von seiner Umgebung abhebt. Marker werden deshalb üblicherweise mit einem weißen Rand gedruckt.

3.4.2 Homographie

Liegen alle betrachteten Punkte in einer Ebene, etwa der Tischebene, vereinfacht sich das Kameramodell aus Gleichung (3.6). Wählt man das Welt-KS so, dass die Ebene $Z = 0$ ist, entfällt die dritte Spalte von $\mathbf{R}$, und die Abbildung zwischen einem Punkt (x, y) der Ebene und seinem Bildpunkt (u, v) ist eine Homographie [7]:

$$\lambda \begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = \mathbf{H} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}, \quad \mathbf{H} \in \mathbb{R}^{3 \times 3}. \quad (3.8)$$

Da $\mathbf{H}$ nur bis auf einen Faktor bestimmt ist, hat sie acht Freiheitsgrade. Jede Punkt-korrespondenz liefert zwei Gleichungen, sodass vier Korrespondenzen, von denen keine drei auf einer Geraden liegen, zur Bestimmung genügen. Beim Direct Linear Transform (DLT) werden die Gleichungen aller Korrespondenzen zu einem linearen Gleichungssystem $\mathbf{A}\mathbf{h} = \mathbf{0}$ für die neun Einträge $\mathbf{h}$ von $\mathbf{H}$ zusammengefasst. Die Lösung ist der rechte Singulärvektor von $\mathbf{A}$ zum kleinsten Singulärwert [7]. Bei mehr als vier Korrespondenzen liefert dasselbe Vorgehen eine Lösung im Sinne der kleinsten Quadrate.

Mit der Umkehrung von $\mathbf{H}$ lässt sich jedes Kamerabild in eine Draufsicht der Ebene umrechnen, in der Längen und Winkel unabhängig von der Kameraperspektive sind. Das gilt jedoch nur für Punkte in der Ebene selbst. Punkte oberhalb der Ebene, etwa die Oberseite eines Objekts, werden in der Draufsicht versetzt abgebildet (Parallaxe), und zwar umso stärker, je höher sie liegen und je weiter sie von dem Punkt direkt unter der Kamera entfernt sind.

3.5 Koordinatentransformationen

Lagen und Bewegungen in der Robotik werden in verschiedenen Koordinatensystemen angegeben, zwischen denen umgerechnet werden muss. Eine Starrkörpertransformation besteht aus einer Drehung $\mathbf{R}$ und einer Verschiebung $\mathbf{t}$ und erhält Längen und Winkel. In homogenen Koordinaten, bei denen einem Punkt eine zusätzliche 1 angehängt wird, lässt sie sich als eine einzige Matrixmultiplikation schreiben. In der Ebene gilt

$$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} \cos \varphi & -\sin \varphi & t_x \\ \sin \varphi & \cos \varphi & t_y \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}. \quad (3.9)$$

Mehrere Transformationen hintereinander ergeben sich als Produkt der Matrizen. Im Raum hat die Matrix die Größe 4×4 mit einer 3×3 -Drehmatrix.

Ist die Transformation zwischen zwei Koordinatensystemen nicht bekannt, lässt sie sich aus Punkten bestimmen, deren Koordinaten in beiden Systemen gemessen wurden. Gesucht sind $\mathbf{R}$ und $\mathbf{t}$, die die Summe der quadratischen Abstände

$$E(\mathbf{R}, \mathbf{t}) = \sum_{i=1}^n \left\| \mathbf{R} \mathbf{p}_i + \mathbf{t} - \mathbf{q}_i \right\|^2 \quad (3.10)$$

zwischen den transformierten Punkten $\mathbf{p}_i$ und ihren Gegenstücken $\mathbf{q}_i$ minimieren. Die Lösung lässt sich geschlossen angeben [10]: Beide Punktmengen werden um ihre Schwerpunkte $\bar{\mathbf{p}}$ und $\bar{\mathbf{q}}$ zentriert, und die Kovarianzmatrix $\mathbf{C} = \sum_i (\mathbf{q}_i - \bar{\mathbf{q}})(\mathbf{p}_i - \bar{\mathbf{p}})^\top$ wird per Singulärwertzerlegung $\mathbf{C} = \mathbf{U} \mathbf{\Sigma} \mathbf{V}^\top$ zerlegt. Dann ist

$$\mathbf{R} = \mathbf{U} \mathbf{D} \mathbf{V}^\top, \quad \mathbf{t} = \bar{\mathbf{q}} - \mathbf{R} \bar{\mathbf{p}}, \quad (3.11)$$

wobei die Diagonalmatrix $\mathbf{D} = \text{diag}(1, \det(\mathbf{U}\mathbf{V}^\top))$ (in der Ebene) sicherstellt, dass eine Drehung und keine Spiegelung entsteht.

3.6 Industrierobotik und Greifen

3.6.1 Kinematik und Koordinatensysteme

Ein Industrieroboter mit sechs Drehgelenken ist eine offene kinematische Kette aus Gliedern, die über Gelenke verbunden sind [11]. Die Stellung aller Gelenke, der Gelenkwinkelvektor, legt eindeutig fest, wo sich das Werkzeug befindet. Die Berechnung dieser Lage aus den Gelenkwinkeln heißt *Vorwärtskinematik*. Für Aufgaben wie das Greifen ist meist die umgekehrte Frage wichtig: Welche Gelenkwinkel führen das Werkzeug an eine gewünschte Lage? Diese *inverse Kinematik* hat im Allgemeinen mehrere Lösungen, und die Steuerung wählt eine davon, meist die nächstgelegene zur aktuellen Stellung.

Für die Beschreibung von Lagen werden mehrere KS verwendet. Das Basis-KS ist fest mit dem Sockel des Roboters verbunden. Das Werkzeug-KS liegt im TCP, dem Arbeitspunkt des Werkzeugs, bei einem Greifer typischerweise zwischen den Greiferbacken. Eine Pose des Werkzeugs besteht aus der Position des TCP im Basis-KS und der Orientierung des Werkzeug-KS, die häufig durch drei Drehwinkel (Roll, Nick, Gier) angegeben wird.

3.6.2 Bewegungsarten

Steuerungen bieten im Wesentlichen zwei Arten an, eine Zielpose anzufahren:

Point-to-Point (PTP)-Bewegung Alle Gelenke fahren von ihrer Start- zu ihrer Zielstellung. Der Weg des TCP im Raum ist dabei nicht festgelegt, dafür ist die Bewegung schnell und frei von Problemen durch singuläre Stellungen. Sie eignet sich für Wege durch freien Raum.

LIN-Bewegung Der TCP bewegt sich auf einer Geraden zur Zielpose. Die Steuerung berechnet dafür laufend die inverse Kinematik. Die Bewegung ist vorhersehbar und eignet sich zum Anfahren und Verlassen von Greif- und Ablagepositionen.

3.6.3 Greifer und Greifstrategie

Industrielle Greifer halten Werkstücke mechanisch mit Backen, über Unterdruck mit Saugnapfen oder magnetisch [11]. Ein Parallelgreifer hat zwei Backen, die sich parallel zueinander schließen, und hält ein Werkstück an zwei gegenüberliegenden Flächen.

Für einen Griff müssen Greifposition und Greiforientierung festgelegt werden. Bei flach liegenden Teilen und einem von oben greifenden Werkzeug reduziert sich das auf die Position in der Ebene, die Greifhöhe und die Drehung des Greifers um die senkrechte Achse. Die Anfahrt erfolgt üblicherweise über eine Vorposition oberhalb des Teils, von der aus der Greifer geradlinig abgesenkt wird, damit er das Teil nicht seitlich verschiebt.

3.6.4 Kollaborative Roboter

Kollaborative Roboter (kollaborativer Roboter (Cobot)s) sind dafür ausgelegt, ohne trennende Schutteinrichtung in der Nähe von Menschen zu arbeiten. Sie begrenzen dazu Kräfte und Geschwindigkeiten und erkennen Kollisionen, etwa über Drehmomentensensoren in den Gelenken [11]. Die Anforderungen an solche Anwendungen sind in Normen festgelegt; ob eine Anwendung sicher ist, hängt aber nicht allein vom Roboter ab, sondern auch von Werkzeug, Werkstück und Bewegungsablauf und muss für jede Anwendung beurteilt werden.

Normen zu Industrie- und kollaborativen Robotern (ISO 10218, ISO/TS 15066) zitieren, falls gewünscht (keine DOI, als Norm aufnehmen).

3.7 Optische Messtechnik und Prüfmerkmale

3.7.1 Beleuchtung

Für die optische Prüfung ist die Beleuchtung oft wichtiger als der Algorithmus. Bei Auflicht befindet sich die Lichtquelle auf der Seite der Kamera und beleuchtet die sichtbare Oberfläche; diffuses Licht vermeidet dabei Glanzstellen und harte Schatten. Bei Durchlicht liegt das Objekt zwischen Lichtquelle und Kamera, und seine Umrisse und Öffnungen erscheinen mit hohem Kontrast. Eine gleichbleibende Beleuchtung macht die Ergebnisse reproduzierbar.

3.7.2 Kantendetektion

Kanten sind Stellen, an denen sich die Helligkeit stark ändert. Sie werden über den Gradienten des Bildes gefunden, der meist mit kleinen Differenzfiltern wie dem Sobel-Operator berechnet wird. Der Kantendetektor nach Canny [12] verbindet mehrere Schritte zu einem robusten Verfahren: Glättung mit einem Gaußfilter, Berechnung von Betrag und Richtung des Gradienten, Ausdünnen der Kanten auf eine Breite von einem Pixel (Non-Maximum Suppression) und eine Schwellwertbildung mit zwei Schwellen (Hysterese). Starke Kanten werden dabei immer übernommen, schwache nur, wenn sie an eine starke Kante anschließen.

3.7.3 Detektion von Kreisen und Ellipsen

Die Hough-Transformation [13] findet geometrische Formen, indem jeder Kantenpunkt für alle Formen „stimmt“, auf denen er liegen könnte. Für einen Kreis $(x-a)^2 + (y-b)^2 = r^2$ ist der Parameterraum dreidimensional; die Einträge eines Akkumulators über (a, b, r) werden für jeden Kantenpunkt erhöht, und Maxima entsprechen den gesuchten Kreisen. Das Verfahren ist robust gegenüber Lücken und Störungen in den Kanten, seine Genauigkeit ist aber durch die Auflösung des Akkumulators begrenzt.

Ist die Kontur eines Kreises oder einer Ellipse bereits bekannt, lässt sich die Form genauer durch Anpassung an alle Konturpunkte bestimmen. Eine allgemeine Kegelschnittgleichung

$$ax^2 + bxy + cy^2 + dx + ey + f = 0 \quad (3.12)$$

wird dabei so gewählt, dass die Summe der quadrierten Abweichungen der Punkte minimal wird. Das Verfahren nach Fitzgibbon, Pilu und Fisher [14] verwendet dafür

die Nebenbedingung $4ac - b^2 = 1$, die garantiert, dass das Ergebnis immer eine Ellipse ist, und führt die Anpassung auf ein verallgemeinertes Eigenwertproblem zurück. Da ein kreisrundes Loch bei leicht schräger Ansicht als Ellipse erscheint, eignet sich eine Ellipsenanpassung auch zur Messung von Kreisen.

3.7.4 Formmerkmale und Template-Matching

Um zu entscheiden, ob ein gefundener Bereich eine bestimmte Form hat, werden einfache Formmerkmale herangezogen. Die Rundheit setzt die Fläche eines Bereichs ins Verhältnis zur Fläche seines kleinsten umschließenden Kreises und ist für einen idealen Kreis gleich 1. Die Rechteckigkeit setzt die Fläche ins Verhältnis zur Fläche des kleinsten umschließenden Rechtecks und ist für ein ideales Rechteck gleich 1.

Ein anderer Ansatz ist das Template-Matching [15]. Ein Referenzbild des gesuchten Merkmals wird über das Bild geschoben und an jeder Position mit dem darunter liegenden Bildausschnitt verglichen, etwa über die normierte Kreuzkorrelation. Die Position mit der höchsten Übereinstimmung ist der Fundort, und die Höhe der Übereinstimmung zeigt, ob das Merkmal vorhanden ist. Das Verfahren ist einfach, reagiert aber auf Änderungen von Größe, Drehung und Beleuchtung gegenüber der Referenz.

3.7.5 Umrechnung in metrische Größen

Längen im Bild werden in Pixeln gemessen. Für eine Angabe in Millimetern wird ein Maßstab benötigt, der aus einer Kalibrierung mit einem Referenzobjekt bekannter Größe stammt, etwa einer Messplatte, oder aus einer bekannten Abmessung des Prüfobjekts selbst. Der Maßstab gilt nur für Objekte im gleichen Abstand zur Kamera, und bei schräger Ansicht unterscheidet er sich in verschiedenen Richtungen. Die erreichbare Genauigkeit hängt von der Auflösung, also der Anzahl Pixel pro Millimeter, und davon ab, wie genau die Kanten lokalisiert werden. Verfahren, die viele Konturpunkte gemeinsam auswerten, wie die Ellipsenanpassung, erreichen dabei Genauigkeiten unterhalb eines Pixels.

3.8 Texterkennung (OCR)

Die optische Zeichenerkennung (Optical Character Recognition (OCR)) wandelt Text in einem Bild in maschinenlesbare Zeichen um. Eine OCR-Pipeline umfasst typischerweise die Vorverarbeitung des Bildes (Kontrastverbesserung, Binarisierung, Ausrichtung), die

Lokalisierung der Textbereiche, die Erkennung der Zeichen und eine Nachbearbeitung, etwa den Abgleich mit einem erwarteten Format oder Wörterbuch.

Klassische Systeme wie Tesseract [16] binarisieren das Bild, suchen zusammenhängende Bereiche, fassen sie zu Zeilen und Wörtern zusammen und erkennen die Zeichen anschließend mit einem Klassifikator. Sie sind auf gut lesbaren, dunklen Text auf hellem Grund, wie in gescannten Dokumenten, ausgelegt.

Lernbasierte Verfahren trennen Textdetektion und Texterkennung in zwei neuronale Netze. Für die Detektion bestimmt etwa CRAFT [17] für jeden Bildpunkt, wie wahrscheinlich er zur Mitte eines Zeichens gehört und wie wahrscheinlich er zwischen zwei zusammengehörigen Zeichen liegt, und setzt daraus Wortbereiche zusammen. Für die Erkennung verarbeitet ein Convolutional Recurrent Neural Network (CRNN) [18] den Textausschnitt zunächst mit einem faltenden Netz (Convolutional Neural Network (CNN)), liest die entstehenden Merkmale anschließend mit einem rekurrenten Netz von links nach rechts und gibt die Zeichenfolge aus, ohne dass die einzelnen Zeichen vorher getrennt werden müssen. Die Bibliothek EasyOCR [19] kombiniert diese beiden Verfahren mit vortrainierten Modellen. Solche Verfahren kommen auch mit Text in natürlichen Szenen, unterschiedlichen Schriftarten und ungleichmäßiger Beleuchtung zurecht.

Ist der erwartete Text bekannt, etwa eine Seriennummer aus einer festen Anzahl von Ziffern, lässt sich die Erkennung auf die zulässigen Zeichen beschränken und das Ergebnis mit einem regulären Ausdruck prüfen. Das verringert Verwechslungen, zum Beispiel zwischen der Ziffer 0 und dem Buchstaben O.

3.9 Lernbasiertes Greifen

Person 4: Überblick lernbasierter Greifansätze, z. B. GG-CNN [20].

4 Systemkonzept

Dieses Kapitel beschreibt das Gesamtsystem, bevor die Kapitel 5 bis 8 die einzelnen Module im Detail behandeln. Ausgehend von den Anforderungen werden der Hardwareaufbau, die Softwarearchitektur mit ihren Schnittstellen, die verwendeten Koordinatensysteme und die schrittweise Inbetriebnahme vorgestellt.

4.1 Anforderungen

Das Projekt ist erfolgreich, wenn ein Bauteil, das von Hand an beliebiger Stelle in den Arbeitsbereich gelegt wurde, ohne weiteren Eingriff erkannt, gegriffen, geprüft und entsprechend dem Prüfergebnis abgelegt wird. Die Prüfmerkmale dienen dabei der Demonstration; es werden keine industriellen Toleranzen angestrebt. Aus dieser Zielsetzung ergeben sich die funktionalen Anforderungen in Tabelle 4.1 und die nicht-funktionalen Anforderungen in Tabelle 4.2.

Tabelle 4.1: Funktionale Anforderungen

Nr.	Anforderung
F1	Das System erkennt ein Bauteil im Arbeitsbereich und bestimmt dessen Position und Orientierung in der Tischebene.
F2	Der Roboter greift das Bauteil an der erkannten Pose und legt es an einer festen Prüfposition ab.
F3	An der Prüfposition werden die Seriennummer ausgelesen, der Lochdurchmesser bestimmt und das Vorhandensein der Kerbe geprüft.
F4	Aus den Prüfmerkmalen wird entschieden, ob ein Gut- oder ein Schlechtteil vorliegt; bei einem Schlechtteil werden die Gründe festgehalten.
F5	Der Roboter nimmt das Bauteil an der Prüfposition wieder auf und legt es auf der Ablage für Gut- bzw. Schlechteile ab.
F6	Der gesamte Ablauf aus F1 bis F5 läuft ohne manuellen Eingriff als Zyklus durch.
F7	Tritt ein Fehler auf, stoppt der Roboter und das System wartet auf eine Quittierung durch den Bediener.

Tabelle 4.2: Nicht-funktionale Anforderungen

Nr.	Anforderung
N1	Robustheit: Erkennung und Prüfung funktionieren bei normalem Raumlicht und leichten Schatten.
N2	Modularität: Erkennung, Robotersteuerung und Prüfung sind voneinander getrennt und über definierte Schnittstellen verbunden; ein Modul lässt sich austauschen, ohne die übrigen zu ändern.
N3	Sicherheit: Der Roboter bewegt sich mit reduzierter Geschwindigkeit und nur innerhalb eines festgelegten Arbeitsraums; ein Stopp ist jederzeit möglich.
N4	Konfigurierbarkeit: Aufbauabhängige Werte wie Posen, Kameraeinstellungen und Prüfgrenzen stehen in einer Konfigurationsdatei und nicht im Programmcode.
N5	Nachvollziehbarkeit: Jeder Zyklus wird mit seinem Ergebnis protokolliert.
N6	Einfachheit: Es werden bewusst einfache, nachvollziehbare Verfahren der klassischen Bildverarbeitung eingesetzt.

4.2 Hardwareaufbau

Der Demonstrator besteht aus einem Roboter mit Greifer, zwei fest montierten Kameras, einem durch Marker begrenzten Arbeitsbereich und drei Ablagepositionen. Gesteuert wird er von einem Rechner, der über Local Area Network (LAN) mit der Robotersteuerung verbunden ist. Abbildung 4.1 zeigt den Aufbau, Tabelle 4.3 fasst die Komponenten zusammen.

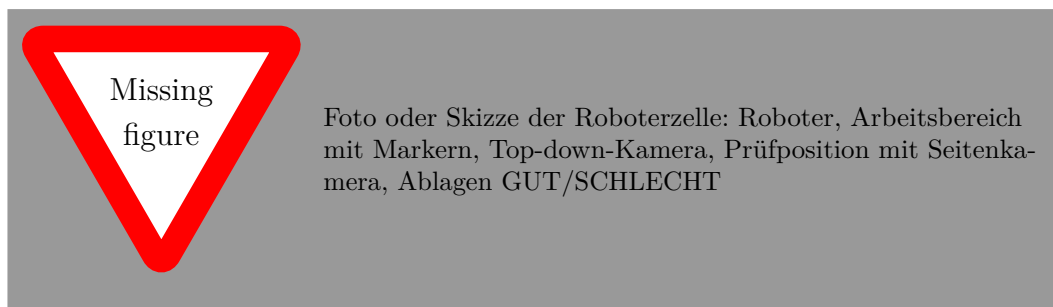


Abbildung 4.1: Aufbau des Demonstrators

Die Fase des Bauteils ist für die Erkennung wichtig: Ohne sie wäre die Draufsicht des Quaders punktsymmetrisch und die Orientierung nur bis auf 180° bestimmbar. Mit der Fase ist die Lage eindeutig, sodass das Bauteil an der Prüfposition immer mit derselben Seite zur Seitenkamera abgelegt werden kann.

Die Form des Bauteils ist bewusst einfach gehalten. Die beiden langen Seitenflächen des Quaders sind parallel, sodass der Parallelgreifer das Bauteil mit seinen Greifbacken direkt an diesen Flächen fassen kann. Eigene Greifbacken oder ein eigener Greifer

Tabelle 4.3: Komponenten des Demonstrators

Komponente	Beschreibung
Roboter	Neura Robotics LARA, sechs Achsen, Steuerungssoftware Version 5.0.8, Ansteuerung über die Python-Schnittstelle NeuraPy [21]
Greifer	Elektrischer 2-Backen-Parallelgreifer Zimmer Group MATCH LWR50L-23-00004-A: Hub 16 mm je Backe, Greifkraft einstellbar bis 500 N, Schließzeit 0,3 s, Wiederholgenauigkeit $\pm 0,02$ mm, Masse 1,4 kg, Ansteuerung über IO-Link [22] Werkzeugdaten in der Steuerung prüfen: Das Werkzeug heißt dort „RobotiQ“. Masse und TCP müssen zum Zimmer-Greifer passen.
Adapter	Zimmer Group MATCH-Robotermodul LWR50F-13-05-A zwischen Roboterflansch und Greifer: Schnellwechsler mit integrierter elektrischer Energieübertragung, Anschlussflansch nach EN ISO 9409-1 (TK 50), Masse 0,35 kg [23]
Top-down-Kamera	über dem Arbeitsbereich, Auflösung 1920×1080 Pixel Kameramodell und Montagehöhe
Seitenkamera	an der Prüfposition, Auflösung 3840×2160 Pixel Kameramodell, Abstand, Beleuchtung
Arbeitsbereich	weißer Rahmen auf dunkler Platte, vier ArUco-Marker an den Ecken (Wörterbuch DICT_4X4_50, IDs 0 bis 3) Innenmaß des Rahmens vermessen
Bauteil	3D-gedruckter Quader mit einer abgeschrägten Ecke (Fase); an einer Seitenfläche ein aufgeklebtes Etikett mit fünfstelliger Seriennummer, ein Durchgangsloch und eine kleine rechteckige Öffnung (Kerbe) Maße aus der Zeichnung
Ablagen	Prüfposition, Ablage GUT und Ablage SCHLECHT als feste Posen mechanischer Anschlag an der Prüfposition?
Rechner	Windows-Laptop mit Python 3.11, LAN-Verbindung zur Robotersteuerung

mussten deshalb nicht entwickelt werden. Voraussetzung ist nur, dass die Breite des Bauteils in den Hub der Greifbacken passt.

Bauteilbreite gegen den Backenhub (16 mm je Backe) und die Öffnungsweite der verwendeten Greifbacken prüfen.

Eine dritte Kamera am Handgelenk des Roboters ist für das lernbasierte Greifen in Kapitel 8 vorgesehen und für den Grundablauf nicht erforderlich.

4.3 Softwarearchitektur

4.3.1 Modulstruktur

Die Software ist in Python geschrieben und in vier Module sowie einen zentralen Orchestrator gegliedert (Abb. 4.2). Der Orchestrator steuert den Ablauf; die Module kennen sich gegenseitig nicht und tauschen ausschließlich festgelegte Datenobjekte aus. Gemeinsam genutzte Funktionen wie Konfiguration, Kamerazugriff und Koordinatentransformationen liegen in einem Paket `common`.

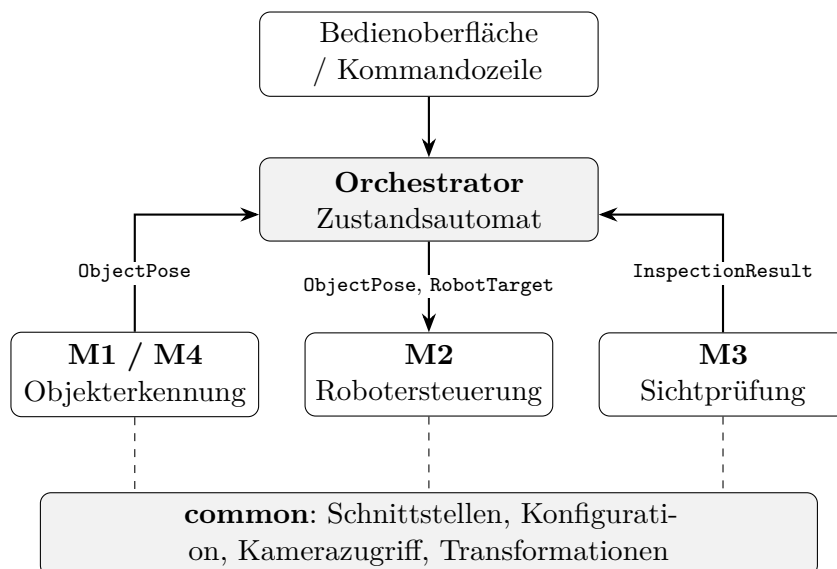


Abbildung 4.2: Modulstruktur der Software

Für jedes Modul gibt es eine abstrakte Schnittstelle und mehrere Implementierungen, darunter immer eine Mock-Implementierung, die zufällige, aber plausible Ergebnisse liefert, ohne auf Hardware zuzugreifen. Welche Implementierung verwendet wird, legt die Konfiguration fest. Dadurch lässt sich jedes Modul isoliert entwickeln, während die übrigen gemockt werden, und die Objekterkennung aus Kapitel 5 kann ohne Codeänderung gegen den lernbasierten Ansatz aus Kapitel 8 getauscht werden.

4.3.2 Ablaufsteuerung

Der Orchestrator bildet einen Arbeitszyklus als endlichen Zustandsautomaten ab (Abb. 4.3). Jeder Zustand ruft genau eine Methode eines Moduls auf; die Zuordnung zeigt Tabelle 4.4. Wird im Zustand **DETECT** kein Bauteil gefunden, kehrt der Automat ohne Fehler nach **IDLE** zurück. Jede Ausnahme eines Moduls führt in den Zustand **ERROR**, in dem der Roboter gestoppt wird. Erst nach einer Quittierung durch den Bediener geht es von dort zurück nach **IDLE**. Details zur Fehlerbehandlung und zur Bedienung folgen in Kapitel 9.

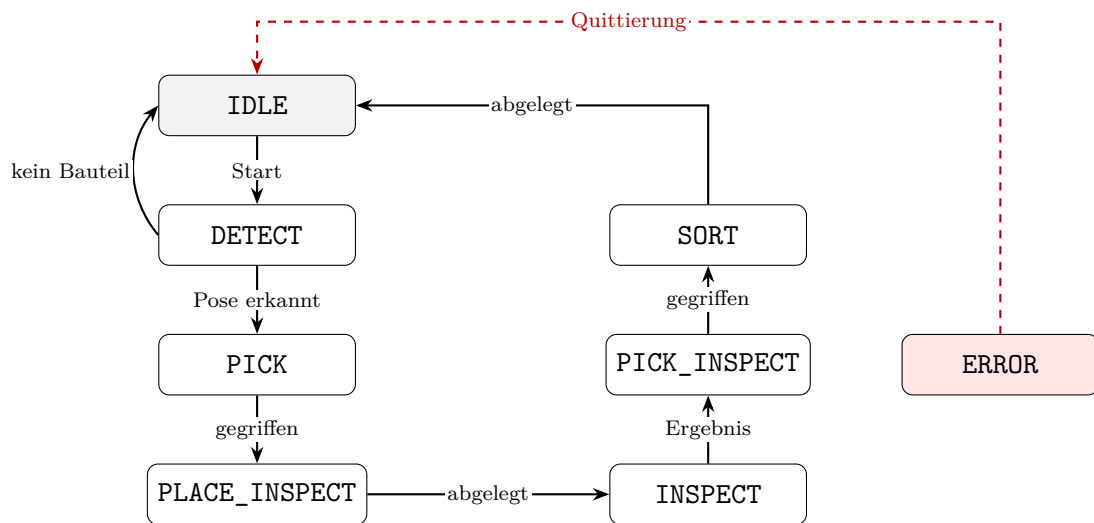


Abbildung 4.3: Zustandsautomat eines Arbeitszyklus. Aus jedem Zustand außer **IDLE** führt ein Fehler in den Zustand **ERROR** (nicht eingezeichnet).

Tabelle 4.4: Zustände des Arbeitszyklus und beteiligte Module

Zustand	Aktion	Modul
IDLE	Warten auf Start bzw. nächsten Zyklus	–
DETECT	Bild aufnehmen, Pose des Bauteils bestimmen	M1 / M4
PICK	Bauteil an der erkannten Pose greifen	M2
PLACE_INSPECT	An der Prüfposition ablegen, Sichtfeld der Seitenkamera freigeben	M2
INSPECT	Seriennummer, Lochdurchmesser und Kerbe prüfen	M3
PICK_INSPECT	Bauteil an der Prüfposition greifen	M2
SORT	Auf Ablage GUT oder SCHLECHT ablegen	M2
ERROR	Roboter stoppen, Fehler protokollieren, auf Quittierung warten	alle

4.3.3 Schnittstellen

Zwischen den Modulen werden drei Datenobjekte ausgetauscht (Tabelle 4.5). Längen werden durchgehend in Millimetern, Winkel in Grad angegeben; die Umrechnung in die Einheiten der Roboterschnittstelle übernimmt die Robotersteuerung.

Tabelle 4.5: Zwischen den Modulen ausgetauschte Datenobjekte

Datenobjekt	Richtung	Inhalt
<code>ObjectPose</code>	M1/M4 $\rightarrow$ M2	Position x , y , z und Drehwinkel θ um die Hochachse im Roboter-KS, Konfidenz, Zeitstempel, optional ein Debug-Bild
<code>RobotTarget</code>	Konfiguration $\rightarrow$ M2	Name und kartesische Pose (Position und drei Drehwinkel) einer festen Roboterposition, z. B. Prüfposition oder Ablage
<code>InspectionResult</code>	M3 $\rightarrow$ Orchestrator	gelesene Seriennummer, gemessener Lochdurchmesser, Kerbe vorhanden, Gesamturteil gut/schlecht und Liste der Gründe für ein Schlechteil

Jedes Modul implementiert eine abstrakte Schnittstelle mit wenigen Methoden (Tabelle 4.6). Fehler meldet ein Modul über eine eigene Ausnahme (Bildverarbeitungs-, Roboter- bzw. Prüffehler), auf die der Orchestrator mit dem Wechsel in den Zustand `ERROR` reagiert.

Tabelle 4.6: Schnittstellen der Module

Schnittstelle	Modul	Methoden
<code>ObjectDetector</code>	M1, M4	<code>detect()</code> : liefert eine <code>ObjectPose</code> oder „kein Bauteil“
<code>RobotController</code>	M2	<code>connect()</code> , <code>home()</code> , <code>pick(pose)</code> , <code>place(target)</code> , <code>stop()</code> , <code>reset()</code>
<code>Inspector</code>	M3	<code>inspect()</code> : liefert ein <code>InspectionResult</code>

4.4 Koordinatensysteme

Damit der Roboter ein Bauteil greifen kann, das die Top-down-Kamera erkannt hat, muss dessen Lage vom Kamerabild in das Koordinatensystem des Roboters überführt werden. Dafür werden drei Koordinatensysteme unterschieden:

Bild-KS Pixelkoordinaten (u, v) im Bild der Top-down-Kamera.

Arbeitsbereich-KS Metrisches KS in der Tischebene mit Ursprung in der inneren linken unteren Ecke des weißen Rahmens (aus Kamerasisht), x -Achse nach rechts, y -Achse nach oben. Die Lage der vier ArUco-Marker ist in diesem KS bekannt.

Roboter-Basis-KS Das KS der Robotersteuerung, in dem alle Bewegungsbefehle angegeben werden.

Da das Bauteil flach auf dem Tisch liegt, genügt für den Übergang vom Bild in den Arbeitsbereich eine Homographie zwischen Bildebene und Tischebene (Abschnitt 3.4). Sie wird bei jeder Aufnahme aus den vier erkannten Markern berechnet. Der Übergang vom Arbeitsbereich in das Roboter-KS ist eine ebene Starrkörpertransformation aus Drehung und Verschiebung (Abschnitt 3.5), ergänzt um die bekannte Tischhöhe. Die Objekterkennung liefert die `ObjectPose` bereits im Roboter-KS, sodass die Robotersteuerung keine Kenntnis über die Kamera benötigt.

Kalibrierung Arbeitsbereich $\rightarrow$ Roboter: Verfahren noch offen, wird ergänzt, sobald es am Aufbau erprobt ist.

4.5 Test- und Inbetriebnahmestrategie

Die Module wurden parallel entwickelt, während Kameras und Roboter nur zeitweise zur Verfügung standen. Die Architektur erlaubt deshalb, jedes Modul in mehreren Stufen zu betreiben, die sich schrittweise der realen Anlage annähern (Tabelle 4.7). Die Stufe wird pro Modul in der Konfiguration oder in der Bedienoberfläche gewählt, sodass sich beliebige Kombinationen bilden lassen, etwa eine reale Kamera zusammen mit einem simulierten Roboter.

Tabelle 4.7: Betriebsstufen der Module

Stufe	Erkennung (M1)	Roboter (M2)	Prüfung (M3)
Mock	zufällige Pose im Arbeitsbereich	Bewegungen werden nur protokolliert, Arbeitsraumgrenzen werden geprüft	zufälliges Prüfergebnis, überwiegend Gutteile
Wiedergabe Simulation	gespeicherte Testfotos –	– Nachbildung der Robotersteuerung mit gleichem Protokoll	gespeicherte Testfotos –
Real	Live-Bild der Top-down-Kamera	realer Roboter	Live-Bild der Seitenkamera

In der Mock-Stufe läuft der vollständige Zyklus ohne jede Hardware und prüft das Zusammenspiel von Orchestrator und Schnittstellen. In der Wiedergabe-Stufe verarbeiten Erkennung und Prüfung gespeicherte Fotos des echten Aufbaus. Da die Bildverarbeitung in beiden Modulen vom Kamerazugriff getrennt ist, durchläuft ein Testfoto denselben Programmcode wie ein Live-Bild. In der Simulationsstufe spricht die Robotersteuerung mit einer Nachbildung der Steuerung, die dasselbe Kommunikationsprotokoll verwendet. So wird der komplette Programmpfad der Robotersteuerung einschließlich der unveränderten Herstellerbibliothek ausgeführt, ohne dass sich ein Roboter bewegt.

Damit lassen sich große Teile der Software ohne Roboter testen. Das ersetzt jedoch nicht die Erprobung an der realen Anlage, bei der Einflüsse wie Beleuchtung, Greifverhalten und Positioniergenauigkeit erst sichtbar werden.

5 Objekterkennung mit der Top-down-Kamera

Die Objekterkennung bestimmt aus einem Bild der Top-down-Kamera, wo das Bauteil im Arbeitsbereich liegt und wie es gedreht ist. Das Ergebnis übergibt sie als `ObjectPose` im Roboter-KS an den Orchestrator (Abschnitt 4.3.3). Da das Bauteil flach auf dem Tisch liegt, genügen dafür die Position (x, y) in der Tischebene und der Drehwinkel θ um die Hochachse; die Höhe ist durch die Tischhöhe festgelegt.

Die Verarbeitung ist als Kette von Einzelschritten aufgebaut (Abb. 5.1). Sie arbeitet ausschließlich auf dem übergebenen Bild und greift selbst nicht auf die Kamera zu. Dadurch lässt sie sich mit gespeicherten Fotos genauso ausführen wie mit Live-Bildern (Abschnitt 4.5).

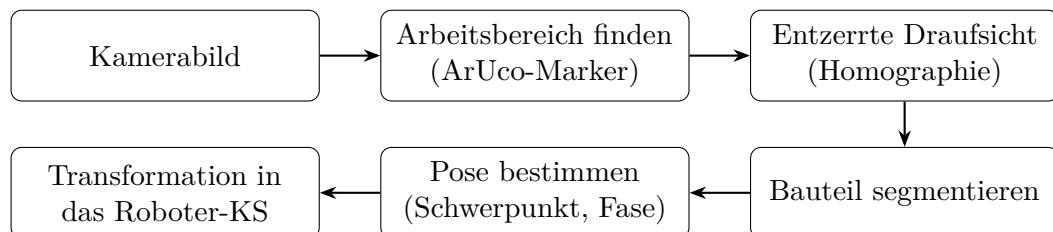


Abbildung 5.1: Verarbeitungskette der Objekterkennung

5.1 Bildaufnahme und Kamerakalibrierung

Die Top-down-Kamera ist fest über dem Arbeitsbereich montiert und liefert Bilder mit 1920×1080 Pixeln. Für reproduzierbare Ergebnisse soll die Belichtung fest eingestellt und nicht automatisch nachgeregelt werden.

Eine Korrektur der Linsenverzeichnung mit einer intrinsischen Kalibrierung nach Zhang (Abschnitt 3.3) ist vorgesehen, aber noch nicht umgesetzt. Die Verarbeitung arbeitet derzeit direkt auf dem unkorrigierten Bild.

Kameramodell, Montagehöhe und Belichtungseinstellung ergänzen. Intrinsische Kalibrierung durchführen und beschreiben, sofern sie umgesetzt wird.

5.2 Detektion des Arbeitsbereichs

Der Arbeitsbereich wird durch einen weißen Rahmen begrenzt, an dessen vier Ecken ArUco-Marker aus dem Wörterbuch DICT_4X4_50 mit den IDs 0 bis 3 angebracht sind [9]. Die Mittelpunkte der Marker sind im Arbeitsbereich-KS vermessen und in der Konfiguration hinterlegt.

Die Marker werden mit dem ArUco-Detektor von OpenCV [2] im Graustufenbild gesucht; die Markerecken werden dabei subpixelgenau verfeinert. Aus den vier Ecken jedes Markers wird dessen Mittelpunkt berechnet. Fehlt einer der vier Marker, bricht die Verarbeitung mit einem Fehler ab, da die Lage des Arbeitsbereichs dann nicht eindeutig bestimmt ist.

Die vier Markermittelpunkte im Bild und ihre bekannten Positionen im Arbeitsbereich bilden vier Punktkorrespondenzen, aus denen eine Homographie zwischen Bildebene und Tischebene berechnet wird (Abschnitt 3.4). Mit ihr wird das Kamerabild in eine entzerrte Draufsicht des Arbeitsbereichs umgerechnet. Diese Draufsicht hat einen festen Maßstab von 2 Pixel/mm; ein Pixel entspricht also 0,5 mm auf dem Tisch, und die Perspektive der schräg oder außermittig montierten Kamera ist ausgeglichen. Alle weiteren Schritte arbeiten in dieser Draufsicht.

Als Rückfallebene kann der Arbeitsbereich auch ohne Marker über die Innenkante des weißen Rahmens gefunden werden. Dazu wird der größte viereckige, innen liegende Rand eines hellen, ungesättigten Bereichs gesucht und seine vier Ecken dienen als Korrespondenzen. Im Regelbetrieb werden die ArUco-Marker verwendet.

5.3 Segmentierung des Bauteils

In der entzerrten Draufsicht muss das Bauteil vom Hintergrund getrennt werden. Die Platte des Arbeitsbereichs ist dunkelgrau, das Bauteil dagegen heller oder farbiger. Die Segmentierung nutzt dazu den HSV-Farbraum (Abschnitt 3.2), in dem Helligkeit (V) und Sättigung (S) getrennt vorliegen.

Anstelle fester Schwellwerte werden die Grenzen relativ zur Platte bestimmt. Da das Bauteil nur einen kleinen Teil des Arbeitsbereichs bedeckt, beschreibt der Median von

Helligkeit und Sättigung über die gesamte Draufsicht die Platte. Ein Pixel gilt als Bauteil, wenn er deutlich heller als die Platte ist oder deutlich stärker gesättigt und dabei nicht zu dunkel. Ändert sich das Raumlicht, verschieben sich Platte und Bauteil gemeinsam, und die Schwelle wandert mit. Außerdem funktioniert die Segmentierung unabhängig von der Farbe des Bauteils, solange es sich in Helligkeit oder Sättigung von der Platte abhebt. Ein globales Schwellwertverfahren wie das nach Otsu (Abschnitt 3.1) wäre eine Alternative, setzt aber ein deutlich zweipfliges Histogramm voraus, das bei einem kleinen Bauteil auf großer Fläche nicht gegeben ist.

Die entstehende Binärmaske wird am Rand des Arbeitsbereichs beschnitten, damit der Rahmen nicht als Bauteil erkannt wird, und anschließend morphologisch bereinigt: Ein Öffnen entfernt einzelne Störpixel, ein Schließen füllt kleine Lücken im Bauteil. Von den gefundenen Konturen werden nur solche mit plausibler Fläche betrachtet; die größte davon ist das Bauteil. Wird keine passende Kontur gefunden, meldet das Modul „kein Bauteil“, und der Orchestrator kehrt ohne Fehler in den Ruhezustand zurück.

5.4 Bestimmung der Pose

Die Position des Bauteils ist der Flächenschwerpunkt der Kontur, der sich aus den Bildmomenten nullter und erster Ordnung ergibt. Für die Orientierung wird das kleinste umschließende Rechteck der Kontur bestimmt; seine lange Seite gibt die Längsachse des Bauteils an.

Die Längsachse allein legt die Orientierung nur bis auf 180° fest, da ein Quader in der Draufsicht punktsymmetrisch ist. Eindeutig wird sie über die abgeschrägte Ecke des Bauteils: An der Fase fehlt dem Bauteil eine Ecke, sodass genau eine Ecke des umschließenden Rechtecks deutlich außerhalb der Kontur liegt. Für jede Rechteckecke wird der Abstand zur Kontur berechnet. Liegt die am weitesten entfernte Ecke mehr als einen festen Anteil der Bauteilbreite außerhalb, gilt die Fase als gefunden, und die Längsachse wird so ausgerichtet, dass sie zur Fase zeigt. Der Winkel ist dann im Bereich 0° bis 360° eindeutig. Wird keine Fase gefunden, wird der Winkel auf 0° bis 180° begrenzt und die Konfidenz der Pose herabgesetzt. Abbildung 5.2 fasst das Vorgehen zusammen.

5.5 Transformation in das Roboterkoordinatensystem

Schwerpunkt und Winkel liegen zunächst in Pixeln der Draufsicht vor. Wegen des festen Maßstabs ergibt sich die Position im Arbeitsbereich-KS durch Division durch

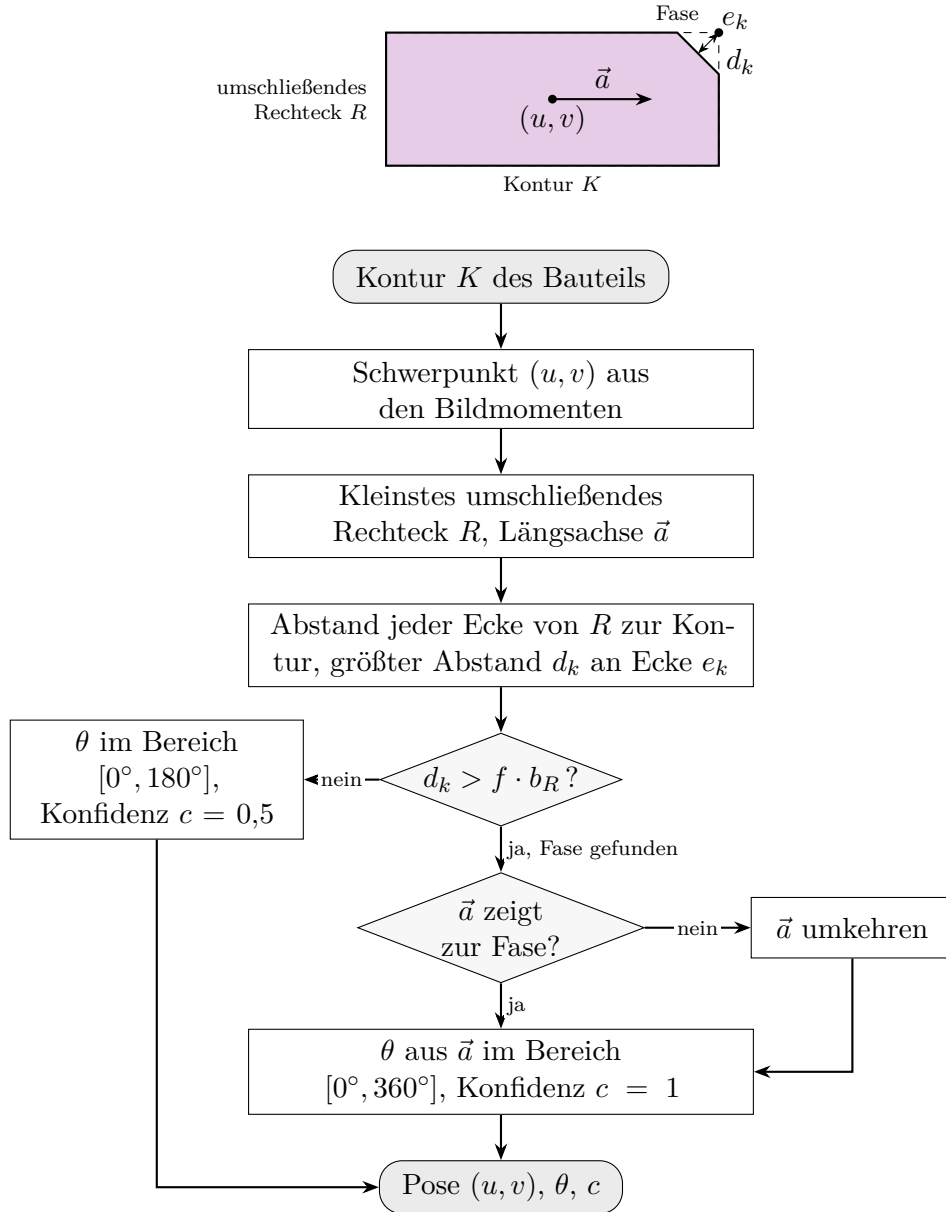


Abbildung 5.2: Posenbestimmung mit Fasenerkennung. Oben: Kontur, umschließendes Rechteck und Abstand d_k der Rechteckecke an der Fase. Unten: Ablauf; b_R ist die Breite von R , f ein fester Anteil.

den Maßstab. Da die Bildzeilen nach unten, die y -Achse des Arbeitsbereichs aber nach oben zeigt, werden dabei die y -Koordinate gespiegelt und das Vorzeichen des Winkels umgekehrt.

Anschließend wird die Pose mit der ebenen Starrkörpertransformation aus Abschnitt 4.4 in das Roboter-KS überführt: Die Position wird gedreht und verschoben, zum Winkel wird der Drehwinkel der Transformation addiert. Als Höhe wird die Tischhöhe eingetragen. Das Ergebnis wird zusammen mit der Konfidenz und einem Debug-Bild als `ObjectPose` zurückgegeben.

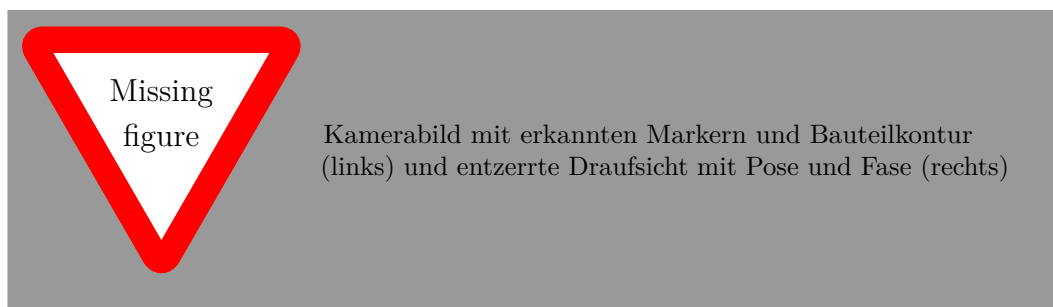


Abbildung 5.3: Zwischenergebnisse der Objekterkennung

Kalibrierung Arbeitsbereich → Roboter: Verfahren noch offen (siehe Abschnitt 4.4).

Parallaxe: Die Kamera sieht die Oberseite des Bauteils, die um die Bauteilhöhe über dem Tisch liegt. Einfluss auf die Position bewerten und ggf. korrigieren.

5.6 Zwischenfazit

Nach den Versuchen ergänzen: Was hat funktioniert, welche Probleme traten auf?

6 Robotersteuerung

Die Robotersteuerung setzt die Bewegungsaufträge des Orchestrators am Roboter um: Sie greift das Bauteil an der erkannten Pose, legt es an der Prüfposition ab, nimmt es dort nach der Prüfung wieder auf und legt es auf der Ablage für Gut- bzw. Schlechteile ab. Nach außen bietet sie dafür die Schnittstelle `RobotController` aus Abschnitt 4.3.3 an. Kenntnis über Kameras oder Prüfergebnisse benötigt sie nicht.

6.1 Roboter und Programmierschnittstelle

Eingesetzt wird ein Neura Robotics LARA, ein kollaborativer Roboter (Cobot) mit sechs Achsen (Abschnitt 3.6). Die Steuerung läuft mit der Softwareversion 5.0.8. Für die Ansteuerung von einem externen Rechner stellt der Hersteller die Python-Bibliothek `NeuraPy` bereit [21]. Sie sendet Befehle als JSON-Nachrichten über eine TCP-Verbindung an die Steuerung und erhält deren Antworten auf demselben Weg.

Kartesische Posen werden in `NeuraPy` als $[x, y, z, \text{roll}, \text{pitch}, \text{yaw}]$ in Metern und Radiant angegeben. Innerhalb des Projekts werden dagegen durchgängig Millimeter und Grad verwendet (Abschnitt 4.3.3); die Umrechnung erfolgt ausschließlich in der Robotersteuerung.

Bevor der Roboter Bewegungen ausführt, wird er vorbereitet: Die Antriebe werden eingeschaltet, die Steuerung wird bei Bedarf vom Handbetrieb in den Automatikbetrieb umgeschaltet, das Werkzeug des Greifers wird ausgewählt, und Geschwindigkeitsbegrenzungen werden gesetzt. Abschließend wird auf der Steuerung ein Bewegungsprogramm gestartet, ohne das keine Bewegungsbefehle angenommen werden. Diese Vorbereitung läuft beim Verbinden und nach jedem Stopp erneut ab.

Für die Bewegungen werden zwei Bewegungsarten genutzt (Abschnitt 3.6):

- **PTP-Bewegung** für die großen Wege zwischen den Stationen. Aus der kartesischen Zielpose werden mit der inversen Kinematik der Steuerung die Gelenkwinkel berechnet, die dann in einer Gelenkbewegung angefahren werden.

- **LIN-Bewegung** für das Absenken auf das Bauteil und das Abheben. Der TCP bewegt sich dabei auf einer Geraden, sodass der Greifer senkrecht auf das Bauteil zufährt.

6.2 Greifpose

Aus der `ObjectPose` der Objekterkennung wird eine Zielpose für den Greifer abgeleitet. Die Position übernimmt x und y direkt; die Höhe ergibt sich aus der Tischhöhe plus einer festen Greifhöhe, die von Bauteil und Greifer abhängt. Der Greifer zeigt immer senkrecht nach unten. Dazu wird die Orientierung der Ruheposition übernommen und nur um die Hochachse des Roboters gedreht, und zwar um den erkannten Bauteilwinkel θ plus einen festen Greifer-Offset. Der Offset von 90° sorgt dafür, dass sich die Greiferbacken quer zur Längsachse schließen, also an den beiden langen Seiten des Bauteils anliegen. Der resultierende Winkel wird auf den Bereich -180° bis 180° normiert.

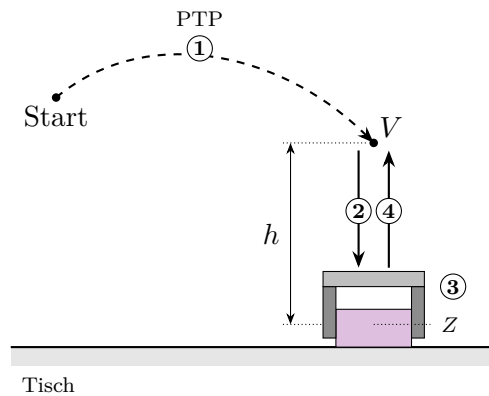
Greifer-Offset und Drehrichtung der Hochachse bei nach unten zeigendem Werkzeug (roll = 180°) am realen Roboter prüfen.

6.3 Greif- und Ablegesequenz

Greifen und Ablegen laufen nach demselben Muster ab: Der Roboter fährt zunächst eine Vorposition oberhalb des Ziels an, senkt sich dann langsam und geradlinig ab, schließt bzw. öffnet den Greifer und hebt wieder auf die Vorposition an (Abb. 6.1). Die Vorposition liegt einen festen Sicherheitsabstand über dem Ziel, sodass der Roboter bei der schnelleren Gelenkbewegung nicht mit dem Bauteil oder dem Tisch kollidiert. Nach dem Schließen und Öffnen des Greifers wird eine kurze Zeit gewartet, bis der Greifer seine Endlage erreicht hat.

Das Greifen an der Prüfposition nutzt dieselbe Sequenz. Statt einer erkannten Pose wird dabei die fest hinterlegte Pose der Prüfposition übergeben, da das Bauteil dort in bekannter Lage liegt.

Greifer-Rückmeldung auswerten: Nach dem Schließen prüfen, ob tatsächlich ein Bauteil gegriffen wurde.



	pick	place
vorab	Z aus erkannter Pose, V und Z prüfen, Greifer öffnen	Z = benannte Pose, V und Z prüfen
①	PTP nach V	
②	LIN nach Z, langsam	
③	Greifer schließen, warten	Greifer öffnen, warten
④	LIN zurück nach V	

Abbildung 6.1: Greif- und Ablegesequenz: Bewegung in der Seitenansicht (links) und Schritte von **pick** und **place** (rechts). Die Vorposition V liegt um den Sicherheitsabstand h über der Zielpose Z . Geprüft wird gegen die Arbeitsraumgrenzen.

6.4 Feste Posen

Neben der erkannten Bauteilpose fährt der Roboter vier feste Posen an (Tabelle 6.1). Sie werden am realen Aufbau im Handbetrieb angefahren (geteacht) und in die Konfiguration übernommen. Die Ruheposition ist als benannter Punkt auf der Steuerung gespeichert und wird direkt in Gelenkkoordinaten angefahren.

Tabelle 6.1: Feste Posen des Roboters

Pose	Zweck
home	Ruheposition außerhalb der Kamerabilder; Start- und Endpunkt
inspection	Ablage an der Prüfposition im Sichtfeld der Seitenkamera
bin_good	Ablage für Gutteile
bin_bad	Ablage für Schlechtteile

Posen am realen Roboter teachen und eintragen. Wiederholgenauigkeit an der Prüfposition beurteilen, da die Sichtprüfung eine reproduzierbare Lage voraussetzt.

6.5 Kalibrierung zwischen Kamera und Roboter

Verfahren noch offen (siehe Abschnitt 4.4).

6.6 Sicherheitskonzept

Die Anlage ist nicht sicherheitszertifiziert und wird nur unter Aufsicht betrieben. Die Robotersteuerung ergänzt die Schutzfunktionen des Cobots um mehrere Softwaremaßnahmen:

Reduzierte Geschwindigkeit Im Testbetrieb sind die globale Geschwindigkeit der Steuerung und die Gelenkgeschwindigkeit auf 20 % begrenzt. Die geradlinige Annäherung an das Bauteil erfolgt zusätzlich mit geringer Geschwindigkeit.

Arbeitsraumgrenzen Vor jeder Bewegung wird geprüft, ob die Zielpose und die Vorphase innerhalb eines festgelegten Quaders im Roboter-KS liegen. Liegt eine Pose außerhalb, wird die Bewegung nicht ausgeführt, und die Robotersteuerung meldet einen Fehler. Dadurch führt auch eine fehlerhafte Pose der Objekterkennung nicht zu einer Bewegung außerhalb des vorgesehenen Bereichs.

Stopp mit Quittierung Ein Stopp beendet das Bewegungsprogramm auf der Steuerung. Danach werden alle Bewegungsbefehle abgelehnt, bis der Bediener den Stopp quittiert und der Roboter erneut vorbereitet wird (Abschnitt 6.1). Das Wiederanlaufen geschieht bewusst nicht automatisch.

Not-Halt Der Hardware-Not-Halt des Roboters ist jederzeit in Reichweite.

6.7 Inbetriebnahme

Für die schrittweise Inbetriebnahme am realen Roboter gibt es ein Prüfprogramm, das die Funktionen einzeln ausführt: Verbindung und Statusabfrage, Anfahren der Ruheposition, Öffnen und Schließen des Greifers, Anfahren einer festen Pose und schließlich ein vollständiger Greif- und Ablegevorgang. Dieselben Schritte stehen in der Bedienoberfläche zur Verfügung (Kapitel 9). Vor dem Einsatz am Roboter wurde der gesamte Programmpfad gegen die Nachbildung der Steuerung getestet (Abschnitt 4.5).

IP-Adresse, Werkzeug und Betriebsartwechsel am realen Roboter verifizieren.

6.8 Zwischenfazit

Nach den Versuchen ergänzen: Was hat funktioniert, welche Probleme traten auf?

7 Sichtprüfung

Die Sichtprüfung beurteilt das Bauteil an der Prüfposition anhand von drei Merkmalen auf seiner Seitenfläche: Sie liest die aufgeklebte Seriennummer, bestimmt den Durchmesser des Durchgangslochs und prüft, ob die Kerbe vorhanden ist. Aus diesen Merkmalen entscheidet sie, ob ein Gut- oder ein Schlechtteil vorliegt, und gibt das Ergebnis als `InspectionResult` an den Orchestrator zurück (Abschnitt 4.3.3).

Wie die Objekterkennung ist auch die Sichtprüfung in eine reine Bildverarbeitung und den Kamerazugriff getrennt, sodass sie sich mit gespeicherten Bildern testen lässt (Abschnitt 4.5). Abbildung 7.1 zeigt den Ablauf.

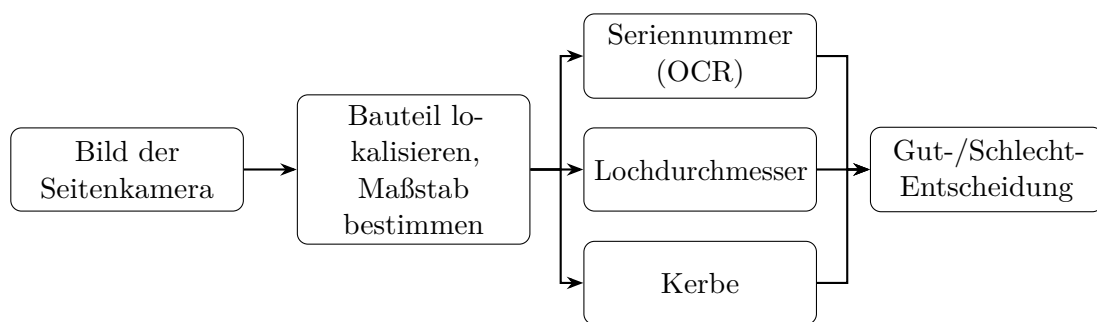


Abbildung 7.1: Ablauf der Sichtprüfung. Die drei Merkmale werden unabhängig voneinander bestimmt und gemeinsam bewertet.

7.1 Aufbau der Prüfstation

Die Seitenkamera ist fest neben der Prüfposition montiert und liefert Bilder mit 3840×2160 Pixeln. Sie blickt möglichst senkrecht auf die Seitenfläche des Bauteils, da eine schräge Ansicht die Längen im Bild perspektivisch verzerren würde. Da die Objekterkennung die Orientierung über die Fase eindeutig bestimmt (Abschnitt 5.4) und der Roboter das Bauteil immer in derselben Lage relativ zum Greifer aufnimmt, liegt es an der Prüfposition stets gleich orientiert, mit Etikett, Loch und Kerbe zur Kamera.

Kameramodell, Abstand und Beleuchtung der Prüfstation ergänzen. Mechanischer Anschlag an der Prüfposition?

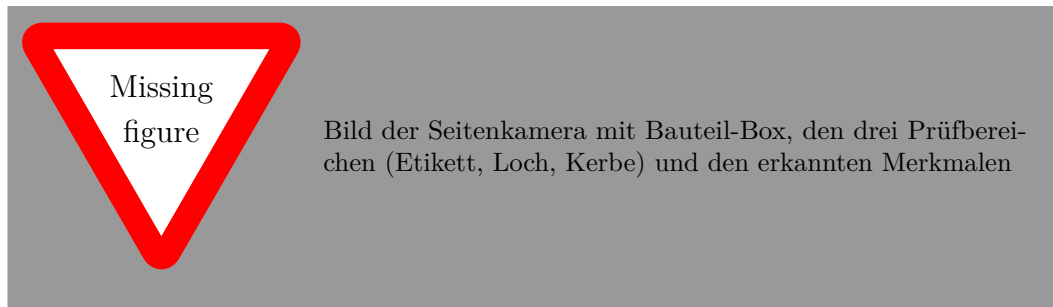


Abbildung 7.2: Seitenansicht des Bauteils mit Prüfbereichen

7.2 Lokalisierung des Bauteils und Maßstab

Zuerst wird das Bauteil im Bild gesucht. Da es sich farblich deutlich vom Hintergrund abhebt, geschieht das über eine Farbschwelle im HSV-Farbraum (Abschnitt 3.2): Pixel mit dem Farbton des Bauteils und ausreichender Sättigung und Helligkeit gehören zum Bauteil. Das weiße Etikett, das Loch und die Kerbe haben nicht die Farbe des Bauteils und erscheinen in der Maske zunächst als Lücken. Ein morphologisches Schließen mit einem Strukturelement in der Höhe des Bauteils füllt diese Lücken, sodass das umschließende Rechteck die gesamte Seitenfläche erfasst.

Die drei Prüfbereiche (Region of Interest (ROI)) für Etikett, Loch und Kerbe sind relativ zu diesem Rechteck festgelegt, jeweils als Bruchteil von dessen Breite und Höhe. Sie passen sich damit automatisch an, wenn das Bauteil etwas näher an der Kamera oder etwas versetzt liegt.

Für die Umrechnung von Pixeln in Millimeter wird die sichtbare Höhe des Bauteils als Maßstab verwendet: Da die tatsächliche Bauteilhöhe bekannt ist, ergibt ihr Verhältnis zur Höhe des Rechtecks in Pixeln den Maßstab in Millimetern pro Pixel. Eine separate Kalibrierung mit einer Messplatte ist dadurch nicht nötig, und der Maßstab bleibt auch bei kleinen Abstandsänderungen richtig. Voraussetzung ist, dass Seitenfläche und Kamera annähernd parallel stehen.

7.3 Auslesen der Seriennummer

Die Seriennummer ist eine fünfstellige Zahl auf dem Etikett. Sie wird mit EasyOCR gelesen [19], einer lernbasierten OCR-Bibliothek, die ohne separate Installation einer OCR-Software auskommt (Abschnitt 3.8). Tesseract [16] wäre eine Alternative gewesen.

Der Etikettbereich wird ausgeschnitten und bei geringer Höhe vergrößert, da die Texterkennung auf sehr kleine Schrift schlechter anspricht. Die Erkennung ist auf Ziffern beschränkt. Aus den erkannten Textkandidaten wird derjenige mit der höchsten Konfidenz gewählt, der dem erwarteten Format entspricht; das Format ist als regulärer Ausdruck für genau fünf Ziffern in der Konfiguration hinterlegt. Liegt die Konfidenz unter einer Mindestschwelle, gilt die Seriennummer als nicht lesbar.

7.4 Vermessung des Lochdurchmessers

Im Lochbereich ist die Seitenfläche des Bauteils hell und farbig. Das Loch hebt sich davon ab, allerdings je nach Beleuchtung unterschiedlich: Die Bohrungswand erscheint dunkel, durch das Loch hindurch kann aber auch heller Hintergrund sichtbar sein. Deshalb wird nicht nach einer bestimmten Farbe des Lochs gesucht, sondern nach allem, was nicht zur hellen Seitenfläche gehört. Als Referenz für die Helligkeit der Fläche dient ein hohes Perzentil der bauteilfarbenen Pixel im Bereich.

Die so gefundenen Regionen werden morphologisch bereinigt und auf eine Mindestgröße gefiltert. Für jede Region wird ihre Rundheit bestimmt, also das Verhältnis ihrer Fläche zur Fläche des kleinsten umschließenden Kreises. Die rundeste Region ist das Loch, sofern ihre Rundheit eine Mindestschwelle überschreitet. An ihre Kontur wird eine Ellipse nach der Methode der kleinsten Quadrate angepasst (Abschnitt 3.7); der Mittelwert der beiden Achsenlängen ergibt den Durchmesser in Pixeln, der mit dem Maßstab aus Abschnitt 7.2 in Millimeter umgerechnet wird. Die Ellipse berücksichtigt, dass ein kreisrundes Loch bei leicht schräger Ansicht elliptisch erscheint. Eine Kreisdetektion mit der Hough-Transformation (Abschnitt 3.7) wäre möglich, liefert den Radius aber nur in der Auflösung ihres Akkumulators, während der Ellipsen-Fit alle Konturpunkte nutzt.

7.5 Prüfung auf Kerbe

Die Kerbe ist eine kleine rechteckige Öffnung am Rand der Seitenfläche. Je nach Licht ist sie dunkler als die Fläche, anders gefärbt oder nur an ihren Kanten zu erkennen. Deshalb werden Kandidaten auf zwei Wegen gesucht:

- **Regionenbasiert:** Pixel, die deutlich dunkler oder deutlich anders gesättigt sind als die umgebende Fläche. Die Referenz für die Fläche ist der Median am Rand des Kerbenbereichs.

- **Kantenbasiert:** geschlossene Kantenzüge aus einer Kantendetektion, für den Fall, dass die Öffnung dieselbe Farbe wie die Fläche hat und sich nur durch ihre Ränder abzeichnet.

Aus allen Kandidaten werden nur solche betrachtet, die vollständig im Kerbenbereich liegen, eine plausible Größe haben und annähernd quadratisch bis mäßig länglich sind. Für jeden wird die Rechteckigkeit bestimmt, also das Verhältnis seiner Fläche zur Fläche des kleinsten umschließenden Rechtecks. Überschreitet der rechteckigste Kandidat eine Mindestschwelle, gilt die Kerbe als vorhanden. Ein Template-Matching gegen ein Referenzbild der Kerbe (Abschnitt 3.7) wäre eine Alternative, reagiert aber im Allgemeinen empfindlicher auf Änderungen von Beleuchtung und Lage.

7.6 Gut-/Schlecht-Entscheidung

Ein Bauteil ist ein Gutteil, wenn alle drei Merkmale erfüllt sind: Die Seriennummer ist lesbar und entspricht dem Format, das Loch wurde gefunden und sein Durchmesser liegt innerhalb der Toleranz um den Sollwert, und die Kerbe ist vorhanden. Andernfalls ist es ein Schlechtteil, und jeder nicht erfüllte Punkt wird als Grund im Prüfergebnis festgehalten (Abb. 7.3). Die Toleranz des Lochdurchmessers dient der Demonstration und entspricht keiner Fertigungsvorgabe.

Bauteilmaße aus der Zeichnung übernehmen: Bauteilhöhe (Maßstab), Soll-Durchmesser und Toleranz des Lochs.

7.7 Zwischenfazit

Nach den Versuchen ergänzen: Was hat funktioniert, welche Probleme traten auf?

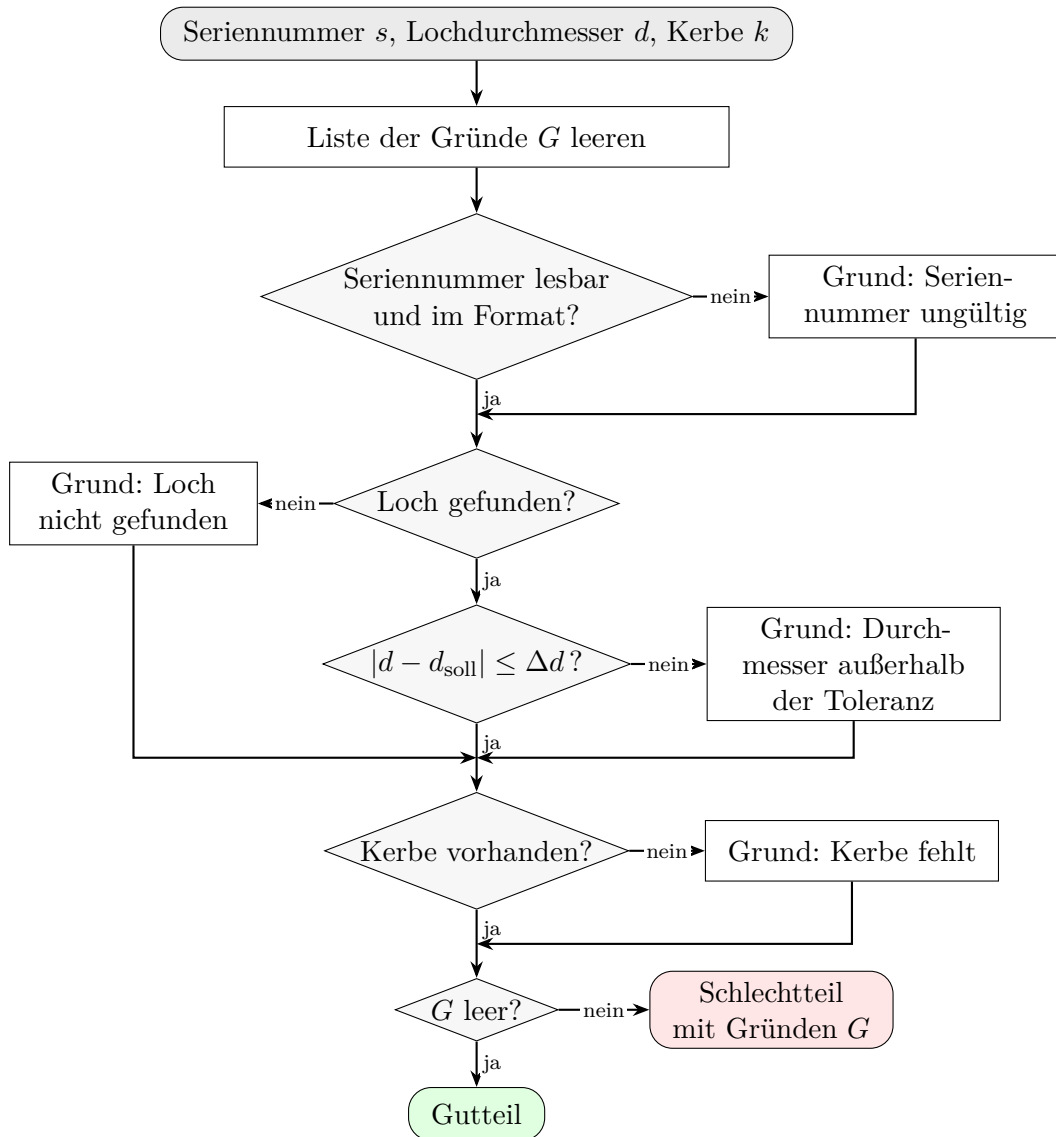


Abbildung 7.3: Gut-/Schlecht-Entscheidung. Jedes nicht erfüllte Merkmal wird als Grund in G festgehalten.

8 KI-basiertes Greifen

8.1 Konzept

8.2 Datensatz und Training

8.3 Integration

8.4 Zwischenfazit

9 Integration und Bedienung

Die Kapitel 5 bis 7 haben die Module einzeln beschrieben. Dieses Kapitel zeigt, wie sie zu einem lauffähigen Gesamtsystem zusammengesetzt werden: wie ein Arbeitszyklus abläuft, wie das System konfiguriert und bedient wird, wie es auf Fehler reagiert und wie die Ergebnisse protokolliert werden.

9.1 Ablauf eines Zyklus

Ein Arbeitszyklus folgt dem Zustandsautomaten aus Abschnitt 4.3. Abbildung 9.1 zeigt die Aufrufe, die der Orchestrator dabei an die Module richtet. Zuerst liefert die Objekterkennung die Pose des Bauteils. Die Robotersteuerung greift das Bauteil, legt es an der Prüfposition ab und fährt anschließend in die Ruheposition, damit der Roboterarm das Sichtfeld der Seitenkamera freigibt. Danach prüft die Sichtprüfung das Bauteil. Abhängig vom Ergebnis nimmt der Roboter das Bauteil wieder auf, legt es auf der Ablage für Gut- oder Schlechteile ab und kehrt in die Ruheposition zurück. Damit ist der Zyklus beendet, und der nächste kann beginnen.

Findet die Objekterkennung kein Bauteil, endet der Zyklus ohne Fehler, bevor sich der Roboter bewegt. Im Dauerbetrieb wiederholt der Orchestrator den Zyklus, bis er angehalten wird oder ein Fehler auftritt.

Da die Module nur über ihre Schnittstellen angesprochen werden, läuft dieser Ablauf unabhängig davon, welche Implementierung eines Moduls gewählt ist. Derselbe Zyklus lässt sich daher vollständig gemockt, mit gespeicherten Testfotos und simuliertem Roboter oder an der realen Anlage ausführen (Abschnitt 4.5).

9.2 Konfiguration

Alle Werte, die vom konkreten Aufbau abhängen, stehen in einer zentralen Konfigurationsdatei im YAML-Format. Dazu gehören:

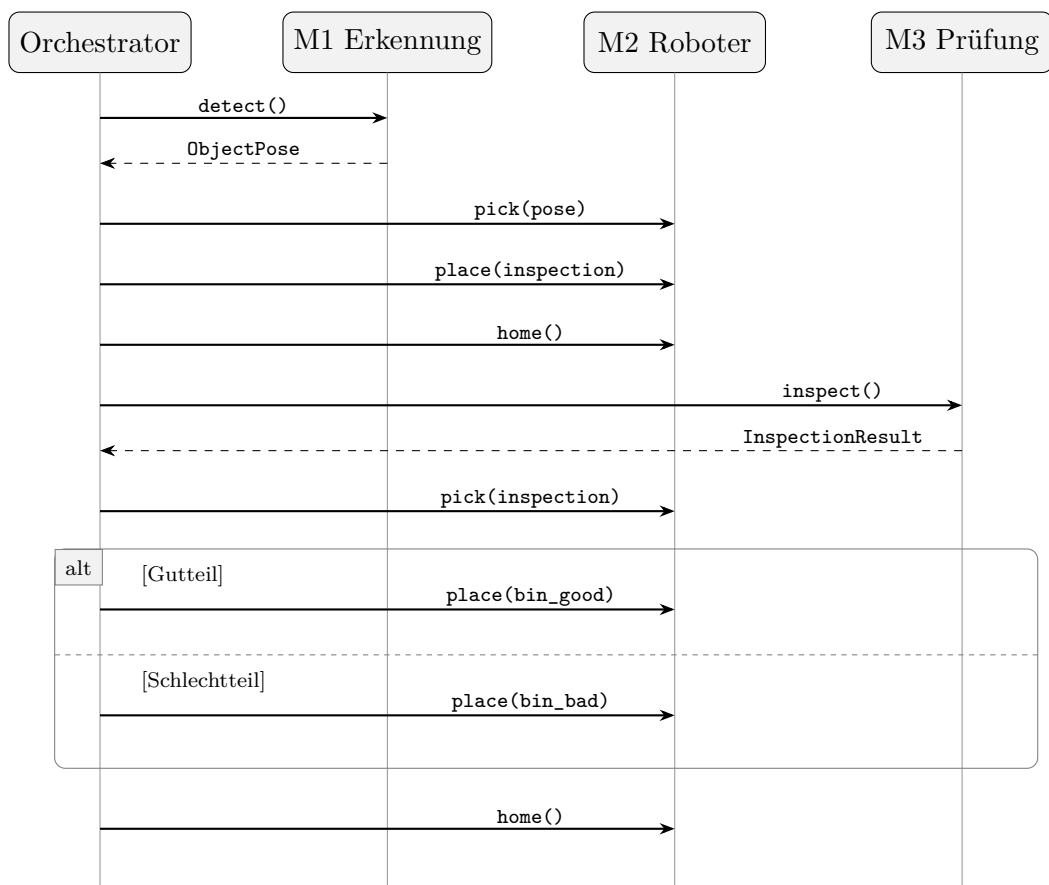


Abbildung 9.1: Aufrufe des Orchestrators an die Module während eines Arbeitszyklus

- Kameraquellen und Auflösungen,
- Größe des Arbeitsbereichs, Wörterbuch und Positionen der ArUco-Marker,
- Schwellwerte der Bildverarbeitung für Segmentierung und Prüfmerkmale,
- Transformation vom Arbeitsbereich in das Roboter-KS,
- Netzwerkadresse, Werkzeug, Geschwindigkeiten, Arbeitsraumgrenzen und feste Posen des Roboters,
- Sollwerte und Format der Prüfmerkmale,
- die standardmäßig verwendete Implementierung jedes Moduls.

Die Module werden über eine gemeinsame Fabrikfunktion erzeugt, die zu einem Modul und einem Variantennamen die passende Implementierung liefert (Tabelle 9.1). Diese Funktion nutzen sowohl der Start über die Kommandozeile als auch die Bedienoberfläche, sodass beide Wege dasselbe System aufbauen.

Tabelle 9.1: Wählbare Varianten der Module

Modul	Variante	Bedeutung
Erkennung	mock	zufällige Pose, keine Kamera
	replay	Objekterkennung auf gespeicherten Testfotos
	camera	Objekterkennung auf dem Live-Bild der Top-down-Kamera
	ai	lernbasierter Ansatz aus Kapitel 8
Roboter	mock	Bewegungen werden nur protokolliert
	sim	Robotersteuerung gegen die Nachbildung der Steuerung
	real	realer Roboter
Prüfung	mock	zufälliges Prüfergebnis, keine Kamera
	replay	Sichtprüfung auf gespeicherten Testfotos
	camera	Sichtprüfung auf dem Live-Bild der Seitenkamera

9.3 Bedienoberfläche

Für Inbetriebnahme und Betrieb gibt es eine grafische Bedienoberfläche (Graphical User Interface (GUI)), die mit dem Framework Qt über dessen Python-Anbindung PySide6 umgesetzt ist. Sie ist in fünf Reiter gegliedert (Tabelle 9.2). Unter den Reitern zeigt ein Protokollfenster laufend die Meldungen aller Module an.

Alle Reiter greifen auf eine gemeinsame Sitzung zu, die Konfiguration, Kameras und Roboteranbindung hält. Eine Kamera oder eine Roboteranbindung wird dadurch nur

Tabelle 9.2: Reiter der Bedienoberfläche

Reiter	Funktion
Ablauf	Auswahl der Varianten, einzelner Zyklus oder Dauerbetrieb, Anzeige des aktuellen Zustands, der Bilder von Erkennung und Prüfung, einer Draufsicht auf den Roboterarbeitsraum sowie einer Tabelle und Zählung der Ergebnisse
M1 Erkennung	Bild aus Kamera, Testfoto oder Datei holen und analysieren, optional fortlaufend; Anzeige des Kamerabilds mit erkannten Markern und der entzerrten Draufsicht mit Pose
M2 Roboter	Verbinden mit Simulator oder realtem Roboter, Status, Greifer öffnen und schließen, feste Posen anfahren, aktuelle Roboterpose als feste Pose übernehmen (Teachen), Stopp und Freigabe
M3 Prüfung	wie M1 für die Seitenkamera; Anzeige der Prüfbereiche und der erkannten Merkmale zum Einstellen der Parameter
Einstellungen	alle Werte der Konfigurationsdatei als bearbeitbarer Baum; Änderungen wirken sofort, Speichern schreibt die Datei unter Erhalt der Kommentare

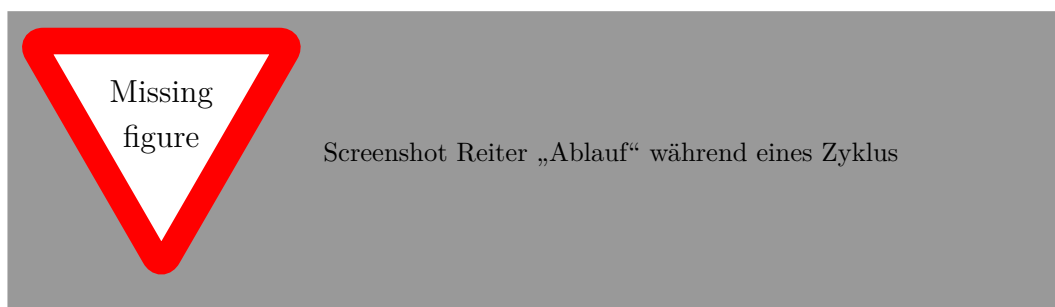


Abbildung 9.2: Bedienoberfläche, Reiter „Ablauf“

einmal geöffnet, auch wenn sie in mehreren Reitern genutzt wird, und Einzeltests in den Modulreitern und der automatische Ablauf verwenden dieselbe Hardware.

Damit die Oberfläche während langer Roboterbewegungen oder Bildverarbeitungsschritte bedienbar bleibt, laufen diese Aufgaben in eigenen Hintergrund-Threads. Alle Befehle an den Roboter werden in einem einzigen Thread strikt nacheinander ausgeführt, sodass sich zwei Bewegungsbefehle nie überschneiden können. Der Zustandsautomat meldet jeden Zustandswechsel an die Oberfläche, die den aktuellen Zustand farbig in der Ablaufkette hervorhebt.

9.4 Fehlerbehandlung und Sicherheit im Ablauf

Jedes Modul meldet Fehler über eine eigene Ausnahme (Abschnitt 4.3.3). Tritt während eines Zyklus eine Ausnahme auf, gleich in welchem Modul, wechselt der Orchestrator in den Zustand `ERROR`, hält die Fehlermeldung zusammen mit dem Zustand fest, in dem sie auftrat, und stoppt den Roboter. Ein Dauerbetrieb wird dabei beendet. Weiter geht es erst, wenn der Bediener den Fehler quittiert: Dann wird der Roboter erneut vorbereitet (Abschnitt 6.1), fährt in die Ruheposition, und der Automat kehrt nach `IDLE` zurück.

Zusätzlich gibt es in der Oberfläche einen Stopp-Knopf, der den Roboter unabhängig vom Zustand des Ablaufs sofort anhält. Er läuft in einem eigenen Thread, damit er nicht hinter einer gerade laufenden Bewegung in der Warteschlange des Roboter-Threads wartet. Der Software-Stopp ergänzt den Hardware-Not-Halt, ersetzt ihn aber nicht.

Bevor der Roboter an der realen Anlage aus der Oberfläche heraus bewegt wird, fragt ein Dialog, ob der Arbeitsraum frei ist und der Not-Halt in Reichweite. Im Dauerbetrieb kann der Ablauf außerdem so angehalten werden, dass der laufende Zyklus noch zu Ende geführt wird.

9.5 Protokollierung

Jeder Zyklus wird als Zeile in einer Comma-Separated Values (CSV)-Datei protokolliert; pro Tag entsteht eine Datei. Eine Zeile enthält Zeitpunkt und Nummer des Zyklus, ob er erfolgreich war, bei einem Fehler dessen Meldung, die Dauer, die erkannte Pose, die Prüfergebnisse (Seriennummer, Lochdurchmesser, Kerbe), das Gesamturteil und die Gründe für ein Schlechterteil. Optional werden zu jedem Zyklus die Zwischenbilder von

Objekterkennung und Sichtprüfung mit eingezeichneten Ergebnissen gespeichert, sodass sich einzelne Zyklen nachträglich nachvollziehen lassen.

Diese Protokolle sind die Datengrundlage für die Evaluation in Kapitel 10.

10 Evaluation

Die Evaluation soll zeigen, ob das Gesamtsystem die Zielsetzung aus Kapitel 1 erfüllt: Ein Bauteil wird ohne Eingriff eines Bedieners erkannt, gegriffen, geprüft und richtig sortiert. Sie ist als Funktionsnachweis angelegt und nicht als Messkampagne zur Genauigkeit einzelner Verfahren. Grundlage sind die Zyklusprotokolle aus Abschnitt 9.5.

10.1 Versuchsaufbau und Vorgehen

Alle Versuche finden an der realen Anlage statt, mit allen Modulen in der Betriebsstufe „Real“ (Abschnitt 4.5). Es werden zwei Versuchsreihen durchgeführt.

Versuchsreihe 1: Wiederholbarkeit. Ein Gutteil wird wiederholt von Hand an zufälliger Position und mit zufälliger Orientierung in den Arbeitsbereich gelegt, und jeweils ein vollständiger Zyklus wird gestartet. Die Reihe zeigt, ob Erkennung, Greifen, Ablegen an der Prüfposition und Prüfung für unterschiedliche Lagen zuverlässig zusammenspielen und das Teil jedes Mal als Gutteil sortiert wird.

Versuchsreihe 2: Aussortierung. Es werden gezielt fehlerhafte Bauteile gefertigt, bei denen jeweils eines der Prüfmerkmale nicht erfüllt ist (Tabelle 10.1). Jedes dieser Teile durchläuft den Zyklus wie in Versuchsreihe 1. Die Reihe zeigt, ob das System die Fehler erkennt und die Teile als Schlechtteil ablegt.

Tabelle 10.1: Fehlerhafte Bauteile für Versuchsreihe 2

Teil	Fehler	Erwartetes Ergebnis
A	Kerbe fehlt	Schlechtteil, Grund „Kerbe fehlt“
B	Lochdurchmesser außerhalb der Toleranz	Schlechtteil, Grund „Durchmesser außerhalb der Toleranz“
C	Etikett mit Seriennummer entfernt	Schlechtteil, Grund „Seriennummer ungültig“

Anzahl der Durchläufe je Versuchsreihe festlegen. Festlegen, ob und welche Kennzahlen ausgewertet werden (z. B. Anteil erfolgreicher Zyklen, Fehler je Zustand, Zyklusdauer).

10.2 Ergebnisse Versuchsreihe 1: Wiederholbarkeit

Ergebnisse nach den Versuchen ergänzen.

10.3 Ergebnisse Versuchsreihe 2: Aussortierung

Ergebnisse nach den Versuchen ergänzen.

10.4 Vergleich klassischer und KI-basierter Ansatz

Person 4: Vergleich der Objekterkennung (Kapitel 5) mit dem lernbasierten Ansatz (Kapitel 8).

11 Fazit und Ausblick

11.1 Zusammenfassung

Nach den Versuchen ergänzen.

11.2 Diskussion

Einordnung der Ergebnisse aus Kapitel 10: Was hat funktioniert, wo lagen die Grenzen, welche Probleme traten auf?

11.3 Ausblick

Offen, ob und welche Punkte hier genannt werden.

Literaturverzeichnis

- [1] Neura Robotics GmbH, *Offline Robot Simulation Setup*, Herstellerdokumentation, 2. Dez. 2024.
- [2] G. Bradski, „The OpenCV Library,“ *Dr. Dobb's Journal of Software Tools*, 2000.
- [3] N. Otsu, „A threshold selection method from gray-level histograms,“ *IEEE Transactions on Systems, Man, and Cybernetics*, Jg. 9, Nr. 1, S. 62–66, 1979. DOI: 10.1109/TSMC.1979.4310076
- [4] S. Suzuki und K. Abe, „Topological structural analysis of digitized binary images by border following,“ *Computer Vision, Graphics, and Image Processing*, Jg. 30, Nr. 1, S. 32–46, 1985. DOI: 10.1016/0734-189X(85)90016-7
- [5] M.-K. Hu, „Visual pattern recognition by moment invariants,“ *IRE Transactions on Information Theory*, Jg. 8, Nr. 2, S. 179–187, 1962. DOI: 10.1109/TIT.1962.1057692
- [6] A. R. Smith, „Color gamut transform pairs,“ *ACM SIGGRAPH Computer Graphics*, Jg. 12, Nr. 3, S. 12–19, 1978. DOI: 10.1145/965139.807361
- [7] R. Hartley und A. Zisserman, *Multiple View Geometry in Computer Vision*, 2. Aufl. Cambridge: Cambridge University Press, 2004. DOI: 10.1017/CB09780511811685
- [8] Z. Zhang, „A flexible new technique for camera calibration,“ *IEEE Transactions on Pattern Analysis and Machine Intelligence*, Jg. 22, Nr. 11, S. 1330–1334, 2000. DOI: 10.1109/34.888718
- [9] S. Garrido-Jurado, R. Muñoz-Salinas, F. J. Madrid-Cuevas und M. J. Marín-Jiménez, „Automatic generation and detection of highly reliable fiducial markers under occlusion,“ *Pattern Recognition*, Jg. 47, Nr. 6, S. 2280–2292, 2014. DOI: 10.1016/j.patcog.2014.01.005
- [10] S. Umeyama, „Least-squares estimation of transformation parameters between two point patterns,“ *IEEE Transactions on Pattern Analysis and Machine Intelligence*, Jg. 13, Nr. 4, S. 376–380, 1991. DOI: 10.1109/34.88573
- [11] B. Siciliano und O. Khatib, Hrsg., *Springer Handbook of Robotics*, 2. Aufl. Cham: Springer, 2016. DOI: 10.1007/978-3-319-32552-1

- [12] J. Canny, „A computational approach to edge detection,“ *IEEE Transactions on Pattern Analysis and Machine Intelligence*, Jg. PAMI-8, Nr. 6, S. 679–698, 1986. DOI: 10.1109/TPAMI.1986.4767851
- [13] R. O. Duda und P. E. Hart, „Use of the Hough transformation to detect lines and curves in pictures,“ *Communications of the ACM*, Jg. 15, Nr. 1, S. 11–15, 1972. DOI: 10.1145/361237.361242
- [14] A. Fitzgibbon, M. Pilu und R. B. Fisher, „Direct least square fitting of ellipses,“ *IEEE Transactions on Pattern Analysis and Machine Intelligence*, Jg. 21, Nr. 5, S. 476–480, 1999. DOI: 10.1109/34.765658
- [15] R. Brunelli, *Template Matching Techniques in Computer Vision: Theory and Practice*. Chichester: Wiley, 2009. DOI: 10.1002/9780470744055
- [16] R. Smith, „An overview of the Tesseract OCR engine,“ in *Ninth International Conference on Document Analysis and Recognition (ICDAR 2007)*, Bd. 2, 2007, S. 629–633. DOI: 10.1109/ICDAR.2007.4376991
- [17] Y. Baek, B. Lee, D. Han, S. Yun und H. Lee, „Character region awareness for text detection,“ in *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019, S. 9357–9366. DOI: 10.1109/CVPR.2019.00959
- [18] B. Shi, X. Bai und C. Yao, „An end-to-end trainable neural network for image-based sequence recognition and its application to scene text recognition,“ *IEEE Transactions on Pattern Analysis and Machine Intelligence*, Jg. 39, Nr. 11, S. 2298–2304, 2017. DOI: 10.1109/TPAMI.2016.2646371
- [19] JaidedAI, *EasyOCR: Ready-to-use OCR with 80+ supported languages*, Version 1.7.2. besucht am 26. Sep. 2026. Adresse: <https://github.com/JaidedAI/EasyOCR>
- [20] D. Morrison, P. Corke und J. Leitner, „Closing the Loop for Robotic Grasping: A Real-time, Generative Grasp Synthesis Approach,“ in *Robotics: Science and Systems XIV*, 2018. DOI: 10.15607/RSS.2018.XIV.021
- [21] Neura Robotics GmbH, *NeuraPy, Release v5.0.8*, Herstellerdokumentation der Python-API, 11. Juli 2025.
- [22] Zimmer Group, *2-Backen-Parallelgreifer LWR50L-23-00004-A*, Produktdatenblatt, Version 1. Adresse: <https://www.zimmer-group.com>
- [23] Zimmer Group, *MATCH-Robotermodul LWR50F-13-05-A*, Produktdatenblatt, Version 1. Adresse: <https://www.zimmer-group.com>